



TUBAF

Die Ressourcenuniversität.
Seit 1765.

Computational Materials Science
Fakultät IV

Personal Programming Project

FEniCSx Implementation of a Two-Dimensional, Thermally Coupled Phase-Field Model for Dendrite Growth

Kartik Suresh Tandel

Matriculation Number: 69110
19.10.2025

Supervised by

Dr.-Ing. Vincent von Oertzen

Professor Incharge

Prof. Dr.-Ing. habil. Bernhard Eidel
Dr.-Ing. Arun Prakash
Dr.-Ing. Stefan Prüger

Abstract

Dendritic growth in pure metals arises from the motion of an anisotropic solid–liquid interface driven by thermal undercooling and latent-heat release. Sharp-interface (Stefan) formulations become cumbersome for complex morphologies, whereas diffuse-interface (phase-field) models describe the interface via a smooth order parameter ϕ coupled to a dimensionless temperature u , enabling robust simulations without explicit front tracking.

This Personal Programming Project implements and verifies a FEniCSx solver for *pure-metal* dendritic solidification based on a variational order parameter and temperature (ϕ – T) formulation with four-fold interfacial anisotropy. The model couples an Allen–Cahn-type evolution to a heat equation with latent-heat source proportional to rate of order parameter ($\partial_t \phi$). The weak forms are discretized with H^1 finite elements and advanced in time using implicit Backward Euler. Nonlinear systems are solved by Newton iterations with assembled Jacobians and direct linear solves (LU) via PETSc backends. The implementation exports XDMF time series for ParaView and supports lightweight live visualization to track interface evolution.

Verification progresses from homogeneous bulk kinetics to curvature-regularized interfaces, to the fully coupled ϕ – u system, and finally to anisotropic dendrite growth. Checks include energy dissipation of the Allen–Cahn gradient flow, an integral enthalpy balance linking $\Delta \int u$ and $\frac{1}{2} \Delta \int \phi$, flat-front tests with constant velocity under uniform driving, and dendrite tip velocity in equiaxed setting. Results confirm stable time integration, physically consistent interface widths, and anisotropy-selected growth directions. The code emphasizes clarity and reproducibility and is designed for extension to 3-D, performance tuning, and additional physics; solute transport and anti-trapping currents are intentionally excluded to isolate thermal effects.

Keywords: Phase-field, Dendrite solidification, FeniCSx

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Objective	4
1.3	Scope (pure metal).	4
2	Theory	4
2.1	Governing equations and Physical Background	4
2.2	Phase-Field Modelling	6
2.3	Homogeneous Phase-Field	7
2.4	Interface Physics	7
2.5	Thermal Coupling with Isotropic Interface	8
2.6	Interfacial Anisotropy	9
3	Numerical Formulation and Implementation.	9
3.1	Weak Form (pure melt)	10
3.2	Implementing Homogeneous Phase-Field	11
3.3	Implementation of Interface Gradient	11
3.4	Implementation of Coupled Phase-Field Model	12
3.5	Implementation of Anisotropy Interface Gradient	15
4	Verification	21
4.1	T1 - Verification of Homogeneous Phase-Field	21
4.2	T2 - Gradient term & Interface tracking (flat front)	24
4.3	T3 - Verification of Temperature and Phase-Field Coupling	28
4.4	T4 - Decoupling Tests	32
4.5	T5 - Verification of Anisotropy	35
5	Choice of Programming Language: Python	38
6	Libraries and Tools	39
7	Comparison to Existing Codes	42
8	Usage of Chatbots	43
9	Manual	44
10	Summary of Proposed Tasks and Achievements	48
11	Conclusion	49
12	Git logs	50

1 Introduction

Dendritic solidification produces tree-like crystalline structures with primary and secondary arms as the solid front advances into an undercooled melt. These branched morphologies govern heat flow, solute segregation, and grain structure, thereby exerting a direct influence on the mechanical performance of materials [1][2]. In additive manufacturing, rapid thermal cycling and steep temperature gradients promote highly anisotropic growth and spatial variability of the microstructure across a build, making predictive modeling particularly important [1].

Classically, dendritic growth is formulated as a sharp-interface moving-boundary (Stefan) problem, in which diffusion is solved separately in each phase and the front velocity follows from an interfacial energy balance [1]. Numerical techniques such as front-tracking, Landau transformations, moving-mesh, or level-set methods can capture interface motion, but they become cumbersome when interfaces curve strongly, branch, impinge, or change topology [1]. Diffuse-interface (phase-field) formulations overcome these challenges by representing the solid-liquid boundary as a narrow transition layer. This allows curved, merging, and branching fronts to evolve naturally on fixed meshes without explicit reconstruction [2]. Under thin-interface scalings, phase-field models remain consistent with the sharp-interface limit and enable quantitative predictions of tip shape and velocity [1].

1.1 Motivation

The present project implements an open, end-to-end phase-field workflow for dendritic solidification using the finite-element framework FEniCSx (dolfinx 0.9.0). The focus is a pure-metal, thermally coupled model incorporating interfacial anisotropy, solved monolithically via a Newton-Krylov scheme with appropriate preconditioning. The phase-field approach has become a powerful, widely used computational tool for understanding and predicting microstructural patterns in materials processing and engineering applications. Implementing and verifying this model within the finite-element framework of FEniCSx covers the complete path from theoretical formulation to verified computation, particularly for a pure-metal model with interfacial anisotropy. This work thereby provides an open-source, extensible baseline that can be leveraged for future studies, such as three-dimensional growth or additive-manufacturing-relevant thermal histories and for rigorous numerical verification within a modern finite-element framework.

1.2 Objective

The main objective of this project was to gain a comprehensive understanding of phase-field simulation and to become familiar with open-source computational frameworks based on the Python programming language. Through the implementation of a dendritic solidification model in FEniCSx, the project aimed to develop both a theoretical foundation and practical programming experience in solving coupled, nonlinear partial differential equations relevant to materials science. In addition to learning the fundamentals of numerical modeling, the project also provided exposure to efficient code structuring, solver configuration, and post-processing techniques in a modern finite-element environment.

Overall, this work establishes an extensible and reproducible baseline for future research on three-dimensional dendritic growth, alloy solidification, and additive-manufacturing-relevant thermal histories. Implementing and validating this model in FEniCSx thus bridges the gap between theory and computation, offering a transparent platform for quantitative studies of microstructural evolution.

1.3 Scope (pure metal).

This project implements a two-field ϕ - T phase-field model for pure-metal solidification with four-fold interfacial anisotropy. The main governing equation consists of following in a non dimensional form. Inspired from [1]

- An Allen–Cahn-type phase-field equation with anisotropic gradient energy
- A heat equation for a nondimensional temperature u including a latent-heat source coupled to $\partial_t \phi$.

2 Theory

2.1 Governing equations and Physical Background

The main governing theory for dendritic solidification is based on the Stefan problem, which models the movement of the solid-liquid interface during phase changes. This problem is extended in dendritic

solidification to account for complex morphologies, such as branching, through phase-field models. These models incorporate additional physics like latent heat release and interfacial anisotropy to simulate the detailed behavior of dendritic growth.

Where equation 1 describes the theoretical framework of heat conduction in both the solid and liquid regions of the pure metal. At the solid-liquid interface, the heat flux from the bulk liquid and bulk solid is balanced by the release of latent heat as the liquid cools and solidifies. This latent heat release drives the movement of the interface, advancing it as new solid material forms from the liquid. Essentially, the interface moves forward as heat is conducted away from the solid-liquid boundary, causing the phase transition from liquid to solid [1]. Further, the balance of the energy is provided in equation 2, which shows that the temperature change at the interface and is affected by certain factors mainly the under-cooling effect due to the surface and kinetics of the interface which intern is covered in the equation 3,

$$\frac{\partial T(\mathbf{x}, t)}{\partial t} = \frac{k}{c_p} \nabla^2 T(\mathbf{x}, t), \quad \mathbf{x} \in \Omega^s \cup \Omega^l. \quad (1)$$

$$\mathbf{v}_n L = k [(\partial_n T(\mathbf{x}))^+ - (\partial_n T(\mathbf{x}))^-], \quad \mathbf{x} \in \Gamma. \quad (2)$$

$$T_m - T_i = \Delta T_\Gamma + \Delta T_\mu = \Gamma_\Gamma \kappa + \frac{\mathbf{v}_n}{\mu_k} \quad \mathbf{x} \in \Gamma. \quad (3)$$

Term Definitions:

- k : Thermal conductivity for pure solid or liquid metal.
- c_p : Volumetric heat capacity for the pure solid or liquid metal.
- T_m : Melting temperature.
- $\mathbf{v}_n$: Interface velocity (normal to the interface).
- L : Latent heat of the metal.
- $\partial T(\mathbf{x})|^{+}$: Temperature gradient normal to the interface in the liquid phase.
- $\partial T(\mathbf{x})|^{-}$: Temperature gradient normal to the interface in the solid phase.
- Γ_g : Gibbs-Thomson coefficient.
- κ : Curvature of the interface.
- μ_k : Interface-attachment coefficient.

The equations from 1 to 3 are transformed into non-dimensional form for use in the phase-field modeling. The temperature T is scaled by the latent heat L and the specific heat c_p of the material, leading to a non-dimensional temperature $u = \frac{c_p(T-T_m)}{L}$, where T_m is the melting temperature. The term $D = \frac{k}{c_p}$ represents the non-dimensional thermal diffusivity in both the solid and liquid phases, where k is the thermal conductivity and c_p is the volumetric heat capacity. This non-dimensionalization simplifies the mathematical model and facilitates the comparison of thermal behavior across different materials under similar conditions. Furthermore, the time scale is adjusted by defining non-dimensional velocity and thermal diffusivity in terms of the system's characteristic values. These transformations reduce the number of variables, making the system more manageable for computational solutions.

Modeling the Stefan problem, which lacks an analytical solution, is a critical challenge, especially when extending it to a free boundary problem. Several methods are available to tackle this, but to obtain an accurate dendritic profile, variationally derived phase-field models are considered the best choice for modeling such complex solidification phenomena [1].

2.2 Phase-Field Modelling

We follow the pure-melt phase-field model of Plapp-Karma [3] as summarized in the reference paper [1]. The state of the system is described by a phase-field order parameter ϕ and a non-dimensional undercooling temperature u . The thermodynamics are encoded by the free-energy isothermal functional which depends on order parameter ϕ and temperature u ,

$$\Pi[\phi, u] = \int_{\Omega} \left(f(\phi, u) + \frac{1}{2} \lambda^2(\mathbf{n}) |\nabla \phi|^2 \right) dV, \quad (4)$$

with a double-well bulk term $f(\phi, u)$ with local minima at $\phi = 1$ and $\phi = -1$, which defines the bulk solid and liquid of the system and an interface term, $\frac{1}{2} \lambda^2(\mathbf{n}) |\nabla \phi|^2$. The figure 1 shows how u can change the direction of solidification.

$$f(\phi, u) = -\frac{1}{2} \phi^2 + \frac{1}{4} \phi^4 + \zeta u \phi \left(1 - \frac{2}{3} \phi^2 + \frac{1}{5} \phi^4 \right) \quad (5)$$

The strength of the interface term is governed by the interface parameter, $\lambda(\mathbf{n})$. This term is positive at the diffuse solid-liquid interface, where there is a nonzero gradient of the order parameter. In regions away from the interface, the interface term becomes zero [1]. The interface parameter, $\lambda(\mathbf{n})$ depends on the interface normal, $\mathbf{n} = \nabla \phi / |\nabla \phi|$.

Term Definitions:

- ζ : Coupling parameter.
- ϕ : Order parameter.
- u : Temperature (non-dimensional)

The governing equations for the phase-field model of solidification are derived by minimizing the above functional 4 with respect to the primary field, ϕ . Similarly, the equations governing the temperature field, u , are obtained by performing a variational derivative of the Lyapunov functional. The non-dimensional enthalpy $U = u - \frac{1}{2} \phi$ accounts for the latent heat coupling. First-order (relaxational) kinetics written for (U, ϕ) and then re-expressed in (u, ϕ) give below:

$$\frac{\partial u}{\partial t} = D \nabla^2 u + \frac{1}{2} \partial_t \phi, \quad \mathbf{x} \in \Omega \quad (6)$$

$$\tau(\mathbf{n}) \frac{\partial \phi}{\partial t} = -\frac{\partial f}{\partial \phi} + \nabla \cdot (\lambda^2(\mathbf{n}) \nabla \phi) + \partial_{\mathbf{x}} \left[|\nabla \phi|^2 \lambda(\mathbf{n}) \frac{\partial \lambda(\mathbf{n})}{\partial (\partial_{\mathbf{x}} \phi)} \right] + \partial_{\mathbf{y}} \left[(|\nabla \phi|^2 \lambda(\mathbf{n}) \frac{\partial \lambda(\mathbf{n})}{\partial (\partial_{\mathbf{y}} \phi)}) \right] \quad \mathbf{x} \in \Omega \quad (7)$$

The anisotropy is introduced via,

$$\lambda(\mathbf{n}) = \lambda_0 a_s(\mathbf{n}), \quad \tau(\mathbf{n}) = \tau_0 a_s^2(\mathbf{n}),$$

$$a_s(\mathbf{n}) = (1 - 3\epsilon_4) \left[1 + \frac{4\epsilon_4}{(1 - 3\epsilon_4)} \frac{(\partial_x \phi)^4 + (\partial_y \phi)^4}{|\nabla \phi|^4} \right] = 1 + \epsilon_4 \cos(m\theta) \quad (8)$$

Term Definitions:

- D : is the thermal diffusivity.
- τ_0 : is the characteristic time scale.
- λ_0 : is the Characteristic length.
- $a_s(\mathbf{n})$: is the Anisotropy parameter.
- Ω : Domain
- ϵ_4 : is the strength of anisotropy

with $m = 4$ for four-fold symmetry and can have any number symmetry $\tan \theta = \frac{\partial \phi}{\partial y} / \frac{\partial \phi}{\partial x}$. Are the key theoretical equation used in defining the anisotropic dendrite formation. [1]

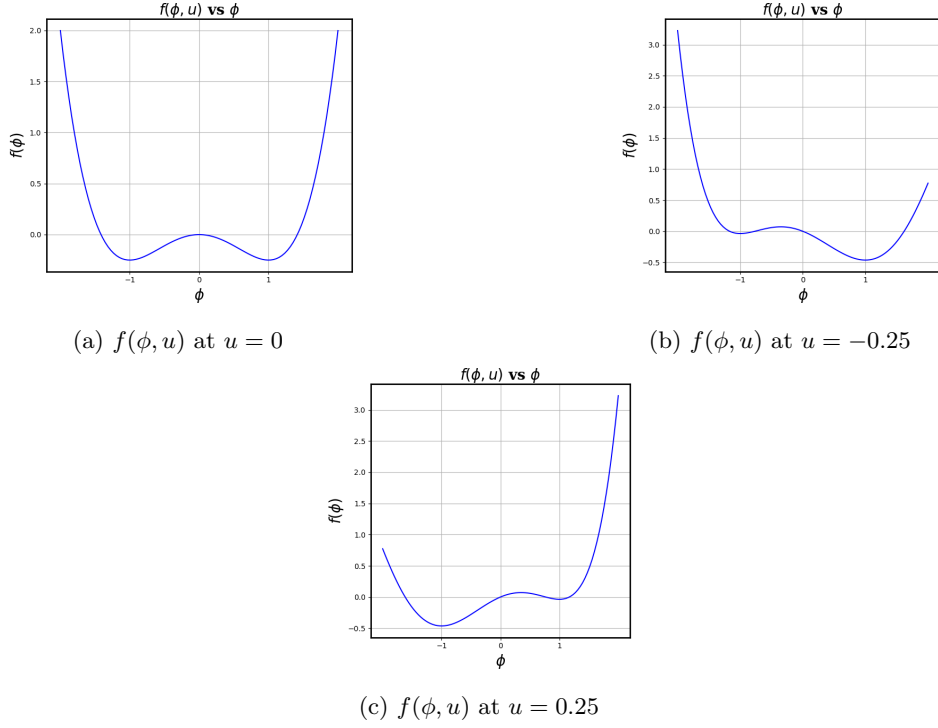


Figure 1: $f(\phi, u)$ vs ϕ for different values of u

2.3 Homogeneous Phase-Field

We start from the bulk free-energy density $f(\phi, u) = g(\phi) + \zeta u h(\phi)$ whose expanded form is shown in equation 5 and neglect spatial gradients ($\nabla\phi = 0$) and temperature evolution (constant u). Relaxational kinetics reduce to the ODE

$$\partial_t \phi = -\frac{\partial f}{\partial \phi}(\phi, u). \quad (9)$$

This model illustrates the double-well structure and how u tilts the wells through the coupling term $\zeta u h(\phi)$. It provides the baseline intuition for metastability and switching thresholds without involving interface curvature or latent heat.

2.4 Interface Physics

We now restore the interfacial term with constant width (no anisotropy yet). The phase-field equation becomes the Allen–Cahn form

$$\tau_0 \partial_t \phi = -\frac{\partial f}{\partial \phi}(\phi, u) + \lambda_0^2 \nabla^2 \phi, \quad (10)$$

where λ_0 sets the diffuse-interface thickness and τ_0 the kinetic scale or characteristic time scale. A circular solid seed embedded in undercooled liquid evolves by curvature-driven motion balanced by the bulk driving; sufficiently small seeds shrink (capillarity effect), while larger seeds grow.

What the gradient term does : In 10, the Laplacian term $\lambda_0^2 \nabla^2 \phi$ comes from the variational derivative of the interface part of the free energy already defined in 4. It penalizes steep changes of ϕ and therefore spreads the sharp solid-liquid boundary into a *diffuse* layer whose thickness is set by λ_0 ; physically, it encodes capillarity [2, 4, 1].

Interface width and surface energy : For the double-well $f(\phi, u)$ in 5, a 1D stationary interface satisfies $\lambda_0^2 \phi'' = \partial f / \partial \phi$. The classic kink solution has a profile $\phi(x) \approx \tanh(x/(\sqrt{2}\xi))$ with a characteristic half-width

$$\xi \propto \sqrt{\frac{\lambda_0^2}{W}},$$

where W is the barrier height of f (from 5). We define the *barrier height* W as the energy difference between the unstable saddle at $\phi = 0$ and either well at $\phi = \pm 1$:

$$W := g(0) - g(\pm 1), \quad g(\phi) := f(\phi, u=0).$$

The corresponding equilibrium surface energy scales like

$$\sigma \propto \sqrt{\lambda_0^2 W}.$$

Thus, increasing λ_0 makes the interface *thicker* and *more expensive* (larger σ). These scalings guide mesh resolution (resolve a few ξ across the interface) and parameter calibration [2, 4].

Curvature $\Rightarrow$ Gibbs–Thomson consistency. Near a curved front, $\nabla^2 \phi = \partial_{\mathbf{nn}} \phi + H \partial_{\mathbf{n}} \phi + \dots$ (mean curvature H). The $H \partial_{\mathbf{n}} \phi$ piece shifts local equilibrium in the diffuse layer and, in the thin-interface limit, recovers the sharp-interface condition in 3 (capillary undercooling $\Gamma \kappa$ plus kinetic term $\mathbf{v}_n / \mu_k$). Choosing (λ_0, τ_0) consistently ensures that the diffuse model 10–11 reproduces the Stefan boundary conditions and quantitative dendrite kinetics [5].

Link to anisotropy : When interface energy depends on orientation, we use $\lambda(\mathbf{n}) = \lambda_0 a_s(\mathbf{n})$ as in 2.2, which effectively makes the width and σ orientation-dependent and selects growth directions (primary arms/side-branching) in 2.6 [1, 5].

2.5 Thermal Coupling with Isotropic Interface

We couple phase evolution equation 10 to temperature via the enthalpy $U = u - \frac{1}{2} \phi$, yielding the pure-melt system with isotropic solidification. How we obtain the equation is seen above in section 2.2. The source to drive the solidification is governed by this equation 11. The source term $\frac{1}{2} \partial_t \phi$ releases latent heat when solid advances ($\partial_t \phi > 0$), locally warming the interface and regulating the tip speed—this is the diffuse analogue of the Stefan balance in 2–3 [1, 5]. The temperature will vary in time and position.

$$\partial_t u = D \nabla^2 u + \frac{1}{2} \partial_t \phi, \quad (11)$$

The source $\frac{1}{2} \partial_t \phi$ represents latent heat release upon solidification. Two useful checks at this stage are: (i) total free energy decreases in time, and (ii) an enthalpy balance between latent heat release and heat diffusion holds (up to numerical error). With these equations, a seed in an undercooled liquid develops a steady tip velocity when geometry and parameters allow.

Physical reading of equation 10–11

- *Heat:* $D \nabla^2 u$ spreads under/over-cooling; the source.
- $\frac{1}{2} \partial_t \phi$ is latent heat release/absorption.
- *Phase:* $-\partial f / \partial \phi$ tilts the double well via u (driving), while $\lambda_0^2 \nabla^2 \phi$ penalizes sharp gradients (capillarity).

Together they produce a diffuse, curvature-sensitive front with kinetics set by τ_0 [2, 4]

Boundary conditions

- Temperature u : prescribed undercooling on inflow boundaries (Dirichlet), or insulated/periodic elsewhere (Neumann/periodic), consistent with your test setups.
- Phase ϕ : no-flux $\nabla \phi \cdot \mathbf{n} = 0$ or periodic, to avoid artificial mass loss/gain of phases.

These BCs ensure the free energy 4 is Lyapunov-decreasing up to discretization error [1].

Interpretation

- **Driving force and latent heat:** The undercooling $u < 0$ tilts $f(\phi, u)$ so that solid is thermodynamically favored ahead of the front. When ϕ advances (solidifies), $\partial_t \phi > 0$ acts as a heat source in 11, locally warming the interface; this feedback regulates the tip velocity.
- **Capillarity and anisotropy.** The gradient term in 10 regularizes the interface and encodes capillarity; the anisotropic parameter $a_s(\mathbf{n})$ biases growth towards anisotropy-preferred directions, selecting the primary arms and enabling side-branching typical of dendrites.
- **Thin-interface consistency.** With appropriate scaling of $\lambda_0, \tau(\mathbf{n}), \lambda(\mathbf{n})$, 11–10 reproduce the Stefan boundary conditions (Gibbs–Thomson + kinetic undercooling) and yield quantitative predictions for dendrite tip shape and velocity.

2.6 Interfacial Anisotropy

To capture dendrite morphology we need an *orientation-dependent* interfacial energy. In the free energy 4 this enters via

$$\frac{1}{2} \lambda^2(\mathbf{n}) |\nabla \phi|^2, \quad \lambda(\mathbf{n}) = \lambda_0 a_s(\mathbf{n}),$$

and, for quantitative consistency in the thin-interface limit, we also scale the kinetics as

$$\tau(\mathbf{n}) = \tau_0 a_s^2(\mathbf{n})$$

Here $\mathbf{n} = \nabla \phi / |\nabla \phi|$ is the unit normal to the interface. Taking the variational derivative gives the anisotropic Allen–Cahn equation (isothermal case):

$$\tau(\mathbf{n}) \partial_t \phi = - \frac{\partial f}{\partial \phi}(\phi, u) + \nabla \cdot (\lambda_0^2 a_s^2(\mathbf{n}) \nabla \phi) + \lambda_0^2 \nabla \cdot \left(|\nabla \phi|^2 a_s(\mathbf{n}) \frac{\partial a_s(\mathbf{n})}{\partial (\nabla \phi)} \right), \quad (12)$$

which is the compact vector form of the divergence corrections written in 7.

Choices for the anisotropy metric $a_s(\mathbf{n})$: We use either of the standard four-fold forms (2D):

$$(\text{angle form}) \quad a_s(\mathbf{n}) = 1 + \epsilon_4 \cos(4\theta), \quad \theta = \arctan 2(\phi_y, \phi_x), \quad (13)$$

$$(\text{quartic, angle-free}) \quad a_s(\mathbf{n}) = (1 - 3\epsilon_4) + 4\epsilon_4 (n_x^4 + n_y^4), \quad \mathbf{n} = \frac{\nabla \phi}{|\nabla \phi|}, \quad (14)$$

which are equivalent to leading order and avoid explicit trigonometry in 14 [2, 4]. The quartic form is convenient numerically; to prevent division by zero at $|\nabla \phi| \approx 0$, replace $|\nabla \phi|$ by $\sqrt{|\nabla \phi|^2 + \eta^2}$ with a small $\eta \sim 10^{-8}$.

Interpretation and expected effects. The factor $a_s(\mathbf{n})$ modulates both the effective interface width and the capillary energy with orientation. For four-fold symmetry and $\epsilon_4 > 0$, growth is biased along the $\langle 10 \rangle$ directions; together with thermal feedback from 11, this selects a steady tip speed and a needle-like shape, i.e. four primary dendrite arms with side-branching [1].

3 Numerical Formulation and Implementation.

The project focuses on modeling dendritic formation in pure metals during solidification, specifically capturing the evolution of dendrite morphology through the phase-field approach and finite element method implementation in 2D. using FEniCSx 0.9.0

The weak forms are posed in H^1 -conforming spaces on two-dimensional meshes and integrated in time with first-order implicit stepping. Nonlinear problems are solved with Newton iterations and the resulting linearized systems are handled by iterative or direct sparse solvers. The baseline problem employs a square domain with a small solid seed embedded in an undercooled liquid; natural (no-flux) boundary conditions are used for both fields unless otherwise stated. The figure 2 show the basic FEM approach we follow in this project.

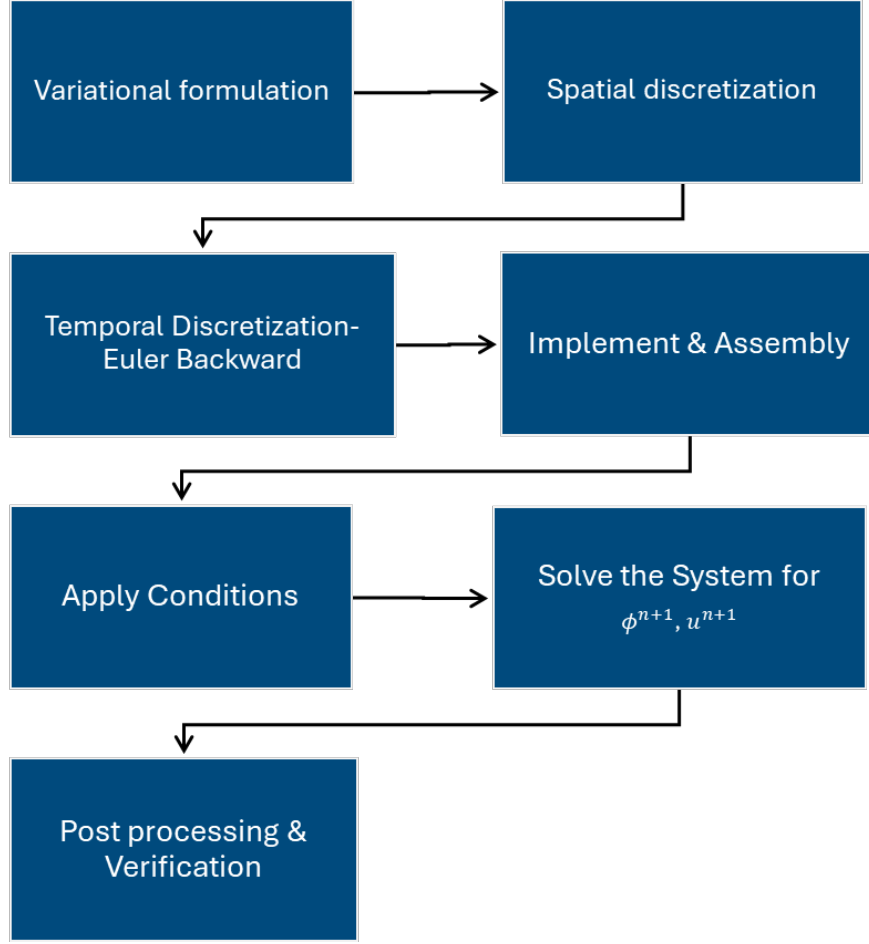


Figure 2: General Methodology followed

3.1 Weak Form (pure melt)

Let $S_u = S_\phi = H^1(\Omega)$, Sobolev space, and test spaces V_u, V_ϕ with homogeneous traces. Find $(u, \phi) \in S_u \times S_\phi$ such that for all $(w_u, w_\phi) \in V_u \times V_\phi$ [1]:

$$\int_{\Omega} w_u \left(\partial_t u - \frac{1}{2} \partial_t \phi \right) dV + \int_{\Omega} D (\nabla w_u \cdot \nabla u) dV = 0, \quad (15)$$

$$\begin{aligned} & \int_{\Omega} w_\phi \left(\tau(\mathbf{n}) \partial_t \phi + \frac{\partial f}{\partial \phi} \right) dV + \int_{\Omega} \lambda^2(\mathbf{n}) (\nabla w_\phi \cdot \nabla \phi) dV \\ & + \int_{\Omega} |\nabla \phi|^2 \frac{\partial w_\phi}{\partial \mathbf{x}} \lambda(\mathbf{n}) \frac{\partial \lambda(\mathbf{n})}{\partial (\frac{\partial \phi}{\partial \mathbf{x}})} dV + \int_{\Omega} |\nabla \phi|^2 \frac{\partial w_\phi}{\partial \mathbf{y}} \lambda(\mathbf{n}) \frac{\partial \lambda(\mathbf{n})}{\partial (\frac{\partial \phi}{\partial \mathbf{y}})} dV = 0 \end{aligned} \quad (16)$$

Homogeneous Neumann BC give no boundary terms in these integrals.

For initial fields, We initialize an under-cooled liquid with a small solid seed:

$$\phi(\mathbf{x}, 0) = \begin{cases} +1, & r(\mathbf{x}) \leq R_0, \\ -1, & r(\mathbf{x}) > R_0, \end{cases} \quad \phi(\mathbf{x}, 0) \approx \tanh\left(-\frac{r(\mathbf{x}) - R_0}{\sqrt{2} \lambda_0}\right), \quad (17)$$

$$u(\mathbf{x}, 0) = u_0 \quad (u_0 < 0 \text{ for undercooling}), \quad (18)$$

where $r(\mathbf{x})$ is the distance from the seed centre. Choose R_0 a few interface thicknesses to avoid capillarity-driven shrinkage.

Boundary condition Zero flux Boundary Conditions are considered on all the boundaries for the fields u and ϕ

$$\nabla \phi \cdot \mathbf{n} = 0, \quad \nabla u \cdot \mathbf{n} = 0 \quad (19)$$

3.2 Implementing Homogeneous Phase-Field

Before starting with the actual task we started with the procedure by implementing the homogeneous phase field. This is a preparatory implementation to check that the phase variable evolves uniformly in space when only the bulk thermodynamics are active. We drop the interfacial (gradient) term and keep the temperature fixed at a prescribed undercooling $u = u_0$. Also the aim is to verify that the function space is attached correctly to the mesh and that the whole domain maintains the same value of $\phi(t)$ as time advances.

With the double-well free energy equation 5, the coupling $\zeta u h(\phi)$ tilts the wells. In our convention liquid $\phi = -1$, solid $\phi = 1$ and an undercooling temperature such as $u_0 = -0.75$, there exists an unstable state near $\phi \approx -2/3$. Initial conditions $\phi_0 < -2/3$ stay in the liquid basin, whereas $\phi_0 > -2/3$ evolve toward the solid basin; in practice we observed that a start slightly above this threshold drives the domain uniformly to the solid state, while values below it remain liquid.

Weak Form: Let $V_\phi \subset H^1(\Omega)$ be the test space with homogeneous traces. The spatially homogeneous evolution is governed by the Allen–Cahn relaxation without interfacial regularization given in equation, 9. The weak form of the isotropic phase field equation is obtained by multiplying the equation by a test function w_ϕ and integrating over the domain Ω . This yields,

$$\int_{\Omega} w_\phi \left(\tau(\mathbf{n}) \partial_t \phi + \frac{\partial f}{\partial \phi} \right) dV = 0 \quad (20)$$

Since the gradient term is absent, no boundary integrals appear and the solution remains spatially uniform if the initial field is uniform.

Initial data: We set $\phi(\mathbf{x}, 0) = -0.6$ (constant in Ω) and keep $u(\mathbf{x}, t) \equiv -0.75$. Any boundary condition on ϕ is immaterial here because $\nabla \phi$ does not enter the equations.

Time discretization used in code. With Backward Euler from $t^n \rightarrow t^{n+1}$, (20) becomes,

$$\int_{\Omega} w_\phi \left(\tau_0 \frac{\phi^{n+1} - \phi^n}{\Delta t} + \frac{\partial f}{\partial \phi}(\phi^{n+1}, u_0) \right) dV = 0, \quad (21)$$

which we solve by a single-field Newton iteration (the residual and Jacobian reduce to element-wise scalars).

The implementation results can be seen below in figure 3, the above mention equation 21 and conditions are used in the coding. The simulation ran in domain size of 100 x 100 with mesh size of $\Delta x = 1$ with time step size, $\Delta t = 0.04$. Like wise, at $u_0 = -0.75$ initialization of $\phi(x, 0) = -0.6$ we can have different initialization of phase at different u_0 . We can calculate by finding the roots of the bulk equation 5 at particular temperature.

Table 1: Material Parameters values used in simulation of homogeneous phase field.

Parameter	Value
$\phi(x, 0)$	-0.6
u_0	-0.75
τ_0	1.0

3.3 Implementation of Interface Gradient

After successfully implementing the homogeneous phase field, the interface gradient term were added with the interface width as a constant, The interface width actually depends on the anisotropy term which is a function of normal direction of the gradient of the order parameter(interface).

In this, The simulation was done by initializing the whole domain to liquid ie $\phi = -1$ and place a circular solid seed at the center mentioned in the initial condition 17. The seed radius R_0 is critical: if R_0 is too small, curvature (capillarity) dominates the bulk driving and the seed dissolves back into the liquid, even for an undercooling as large as $u_0 = -0.75$. In practice, shrinkage occurs when $R_0 \lesssim \lambda_0$ (the diffuse-interface width); the strong interface curvature then overcomes the thermodynamic tendency to solidify and the front recedes rather than advances.

Weak form: Find $\phi \in H^1(\Omega)$ such that for all $w_\phi \in H^1(\Omega)$, The weak form of the isotropic phase field equation is obtained by multiplying the equation by a test function w_ϕ and integrating over the domain

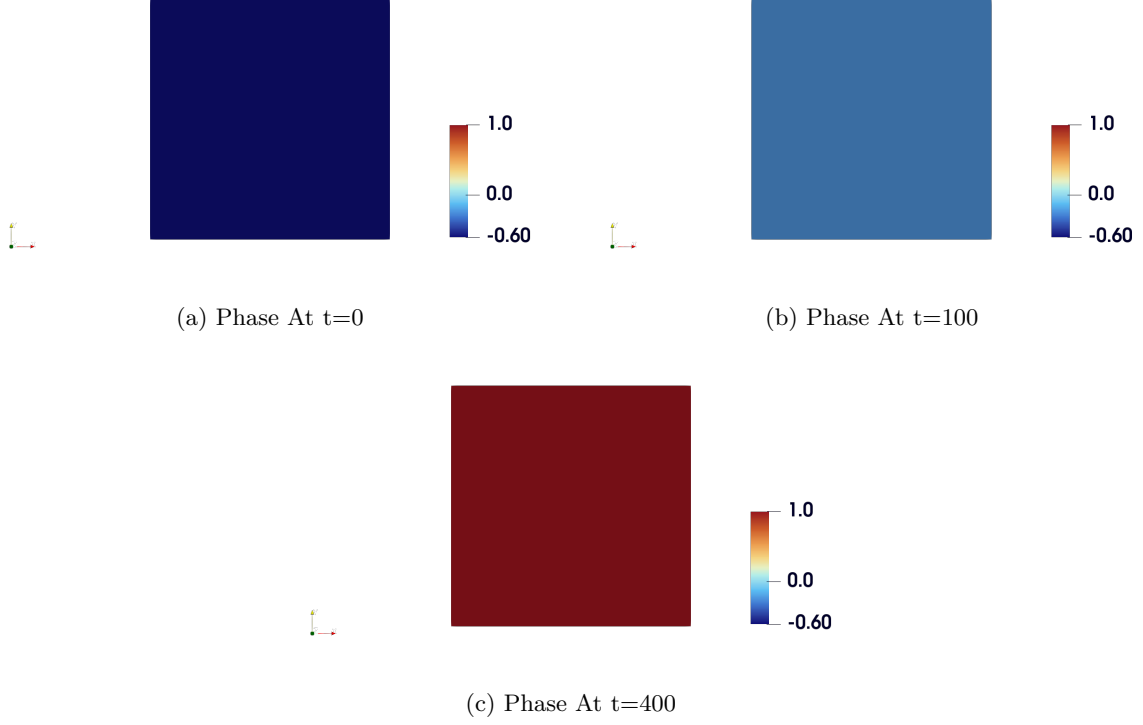


Figure 3: Homogeneous Phase evolution at different simulation time, t

Ω . This yields,

$$\int_{\Omega} w_{\phi} \left(\tau_0 \partial_t \phi + \partial_{\phi} f(\phi, u) \right) dV + \int_{\Omega} \lambda_0^2 \nabla w_{\phi} \cdot \nabla \phi dV = 0. \quad (22)$$

where w_{ϕ} is the test function for the order parameter. The Neumann condition 19 eliminates boundary terms from integration by parts. The initial condition is taken from the 17.

Time discretization: For $t^n \rightarrow t^{n+1}$ with $\Delta t = t^{n+1} - t^n$, seek ϕ^{n+1} such that,

$$\int_{\Omega} w_{\phi} \left(\tau_0 \frac{\phi^{n+1} - \phi^n}{\Delta t} + \partial_{\phi} f(\phi^{n+1}, u_0) \right) dV + \int_{\Omega} \lambda_0^2 \nabla w_{\phi} \cdot \nabla \phi^{n+1} dV = 0. \quad (23)$$

The simulation results can be viewed below in the figure 3 which was simulated on the domain size of 100×100 with mesh size of $\Delta x = 2$ and with time step size, $\Delta t = 0.04$. All the simulation in this task were ran on the same attributes and parameter, given in table 3

In figure 5, at different temperature u_0 we observe the interface moving at different velocities, lesser the undercool temperature faster the interface moves, this tells us that how the rate of solidification depends on the temperature. The time taken to solidify whole domain is given in table 2

Table 2: Wall-clock time vs. initial constant undercooling.

Temperature	Time (s)
-0.75	89.59
-0.50	145.48
-0.25	276.16

3.4 Implementation of Coupled Phase-Field Model

The objective of this task is to couple the temperature partial differential equation (PDE) with the isotropic phase-field model equation 10, omitting the anisotropy term. In this coupled system, the temperature field u becomes a variable that evolves both in space and time, influenced by the phase transition. The temperature, u , in coupling with ϕ drives the solidification process.

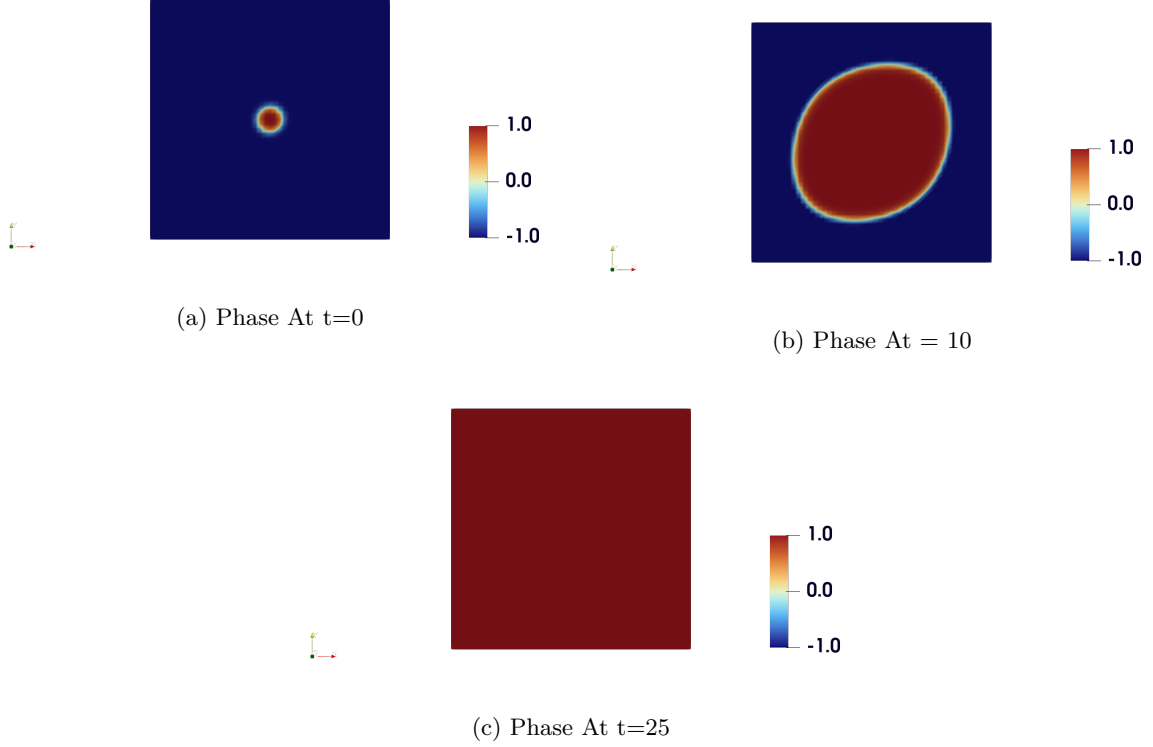


Figure 4: Isotropic phase evolution at constant $u = -0.75$ at different simulation time, t

In this approach, the solidification front is governed by the latent heat term, which is proportional to the rate of change of the order parameter $\frac{\partial \phi}{\partial t}$. As the phase evolves and the material solidifies, latent heat is released, providing the necessary energy to advance the solidification front, thereby promoting the phase change from liquid to solid.

The governing equation for temperature evolution is given in equation 11 with boundary condition given in 19. The weak form of the temperature equation is obtained by multiplying the equation by a test function w_u and integrating over the domain Ω . This yields,

$$\int_{\Omega} w_u \left(\frac{\partial u}{\partial t} - D \nabla^2 u - \frac{1}{2} \frac{\partial \phi}{\partial t} \right) dV = 0, \quad (24)$$

where w_u is the test function for the temperature.

For time integration, we use the backward Euler method. Discretizing the time derivative using the backward Euler scheme, we get:

$$\int_{\Omega} w_u \left(\frac{u^{n+1} - u^n}{\Delta t} - D \nabla^2 u^{n+1} - \frac{1}{2} \frac{\partial \phi^{n+1}}{\partial t} \right) dV = 0, \quad (25)$$

where Δt is the time step size, u^{n+1} and u^n represent the temperature at time steps $n + 1$ and n , respectively. This discretization ensures stability and consistency in the numerical solution.

The term $\frac{\partial u}{\partial t}$ represents the change in temperature with time, while $\nabla^2 u$ represents the diffusion of heat. The phase-field term $\frac{\partial \phi}{\partial t}$ accounts for the latent heat release during phase change, which influences the solidification process.

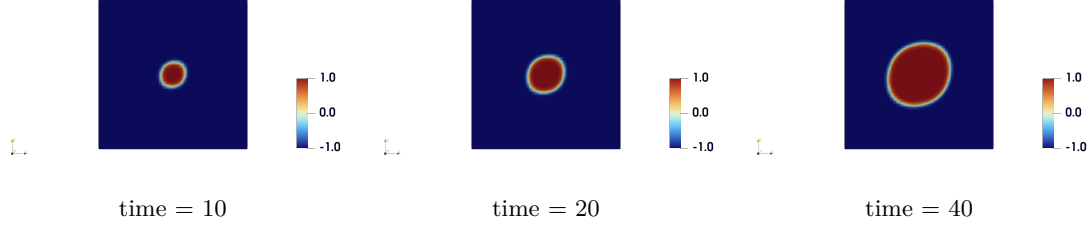
Therefore we have the time integrated, phase field and temperature equation given below.

$$\int_{\Omega} w_{\phi} \left(\tau_0 \frac{\phi^{n+1} - \phi^n}{\Delta t} + \partial_{\phi} f(\phi^{n+1}, u_0) \right) dV + \int_{\Omega} \lambda_0^2 \nabla w_{\phi} \cdot \nabla \phi^{n+1} dV = 0. \quad (26)$$

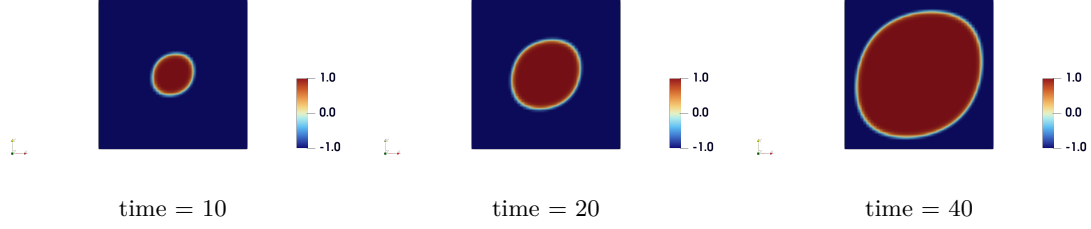
$$\int_{\Omega} w_u \left(\frac{u^{n+1} - u^n}{\Delta t} - D \nabla^2 u^{n+1} - \frac{1}{2} \frac{\partial \phi^{n+1}}{\partial t} \right) dV = 0, \quad (27)$$

The coupling between the temperature field u and the phase-field order parameter ϕ in the code is implemented in the following way,

(1) $u_0 = -0.25$



(2) $u_0 = -0.50$



(3) $u_0 = -0.75$

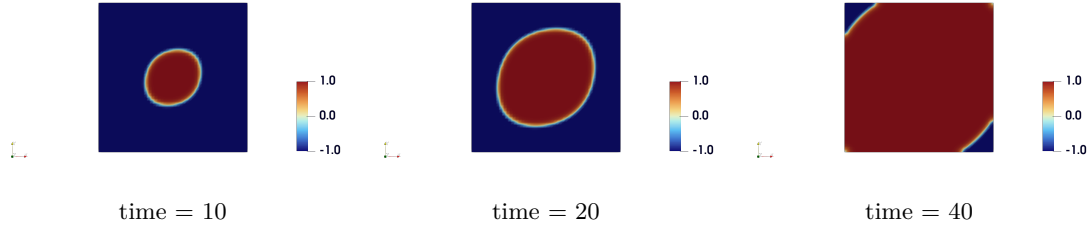


Figure 5: Interface evolution for different undercoolings u_0 and simulation time, t

1. **Phase-Field Update:** The order parameter ϕ evolves according to its governing equation, which is coupled with the temperature field. As the temperature evolves, the latent heat term $\frac{1}{2} \frac{\partial \phi}{\partial t}$ in the temperature equation is updated based on the rate of change of ϕ .
2. **Temperature Update:** Similarly, the temperature u is updated in time, and its change is influenced by the phase-field equation. The temperature is coupled to the phase transition, meaning that as the material solidifies (i.e., as ϕ moves towards solid), the temperature field u evolves.
3. **Implementation in Code:**

- In the finite element code, both ϕ and u are solved together at each time step. The phase-field equation provides the evolution of ϕ , while the temperature equation updates u , can visualized in flowchart 8.
- The weak form of both equations is implemented in the FEniCSx framework, where the system is assembled and solved using Newton method for the nonlinear equations.
- At each time step, the solution for ϕ^{n+1} and u^{n+1} is computed simultaneously by solving the coupled system.

The code implements these equations by defining the weak forms for both the phase-field and temperature equations

The phase-field evolution influences the temperature field in the following ways:

- As the order parameter ϕ evolves towards solidification (solid regions with $\phi \approx 1$ and liquid regions with $\phi \approx -1$), the latent heat released due to phase transition affects the temperature distribution in the domain.
- The temperature gradient drives the solidification front, with the latent heat acting as a source term that influences the growth of the solid phase.

Table 3: Input parameters for the phase-field model of solidification of a pure metal.

Non-dimensional parameter	Numerical value
Thermal diffusivity, D	1.0
Anisotropy strength, ϵ_4	0.05
Characteristic length-scale, λ_0	1.0
Characteristic time-scale, τ_0	1.0
Coupling parameter, ξ	1.60

Thus, the interaction between ϕ and u leads to a coupled, time-dependent system where both fields evolve in tandem as the material undergoes solidification.

The simulation results can be viewed below in the figures, the figure 6 shows the evolution of order parameter or phase and figure 7 shows how the evolution is going towards the equilibrium temperature, These were simulated on the domain size of 200×200 with mesh size of $\Delta x = 1.5$ with $\Delta t = 0.04$, the simulation in this task used the attributes and parameter, given in table 3 and initial condition stated in section 3.1

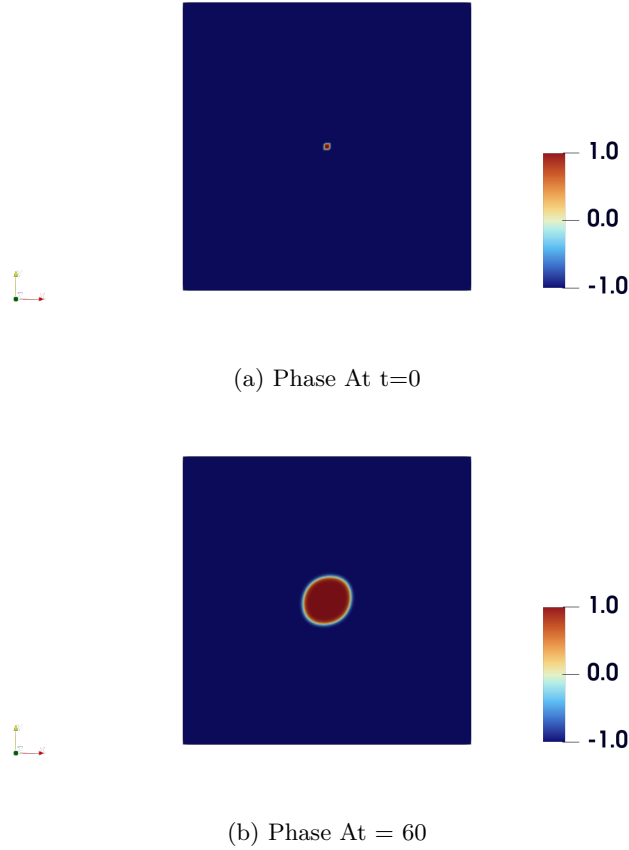
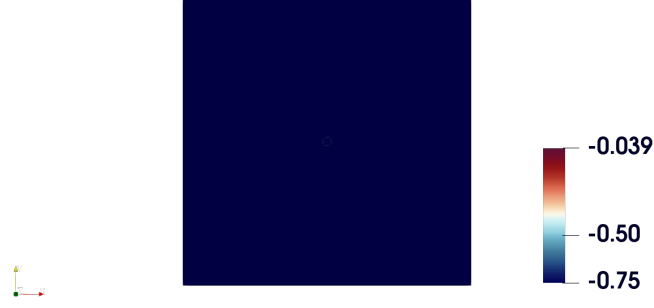


Figure 6: Simulation result of phase evolution in coupled system at simulation time, t

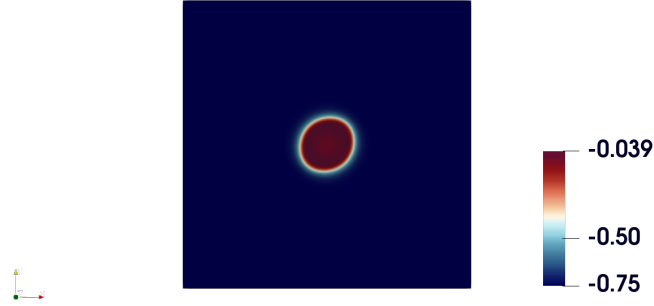
3.5 Implementation of Anisotropy Interface Gradient

In this task, we implemented an anisotropy term to govern the dendritic shape during solidification. The anisotropy term is crucial as it directs the solidification front along specific directions, driven by the minimization of energy. We focused on a four-fold anisotropy model, which results in dendritic arms branching symmetrically in the favorable directions.

Due to the high computational cost of simulating the entire dendritic structure, we were only able to observe the initial branching of the dendritic arms. While the model shows the expected initial growth along the four-fold symmetry directions, capturing the full growth requires further computational



(a) Phase At t=0



(b) Phase At = 60

Figure 7: Simulation result of non dimensional temperature, u evolution in coupled system at simulation time, t

resources.

Implicit Time Integration of the Weak Form:

For the phase-field equation, the anisotropy term also requires implicit time discretizations: we have the thermally couple Isotropic model ready now we need the anisotropy part to complete the whole Phase field of dendrite solidification so,

Additionally, the anisotropic parameter $\lambda(\mathbf{n})$ requires further discretization:

$$\int_{\Omega} |\nabla \phi^{n+1}|^2 \frac{\partial w_{\phi}}{\partial \mathbf{x}} \lambda(\mathbf{n}) \frac{\partial \lambda(\mathbf{n})}{\partial (\partial_{\mathbf{x}} \phi^{n+1})} dV + \int_{\Omega} |\nabla \phi^{n+1}|^2 \frac{\partial w_{\phi}}{\partial \mathbf{x}} \lambda(\mathbf{n}) \frac{\partial \lambda(\mathbf{n})}{\partial (\partial_{\mathbf{y}} \phi^{n+1})} dV \quad (28)$$

In this formulation, $\lambda(\mathbf{n})$ introduces anisotropy by modulating the growth direction of the dendrite. This term adjusts the energy landscape such that dendrites grow along the most favorable directions, as governed by the four-fold symmetry in this case.

The anisotropy parameter a_s is given, after simplifying the equation 8 we have,

$$\mathbf{a}_s = (1 - 3\epsilon_4) + 4\epsilon_4 (n_x^4 + n_y^4)$$

where ϵ_4 is the strength of the anisotropy, and n_x and n_y are the components of the normalized gradient $\mathbf{n}$. The normal gradient components are derived from the phase-field gradient:

$$n_x = \frac{\partial_{\mathbf{x}} \phi^{n+1}}{|\nabla \phi^{n+1}|}, \quad n_y = \frac{\partial_{\mathbf{y}} \phi^{n+1}}{|\nabla \phi^{n+1}|}$$

The magnitude of the gradient is given by:

$$g_n = |\nabla \phi^{n+1}|^2 = \sqrt{n_x^2 + n_y^2}$$

We now compute the derivatives of a_s with respect to n_x and n_y .

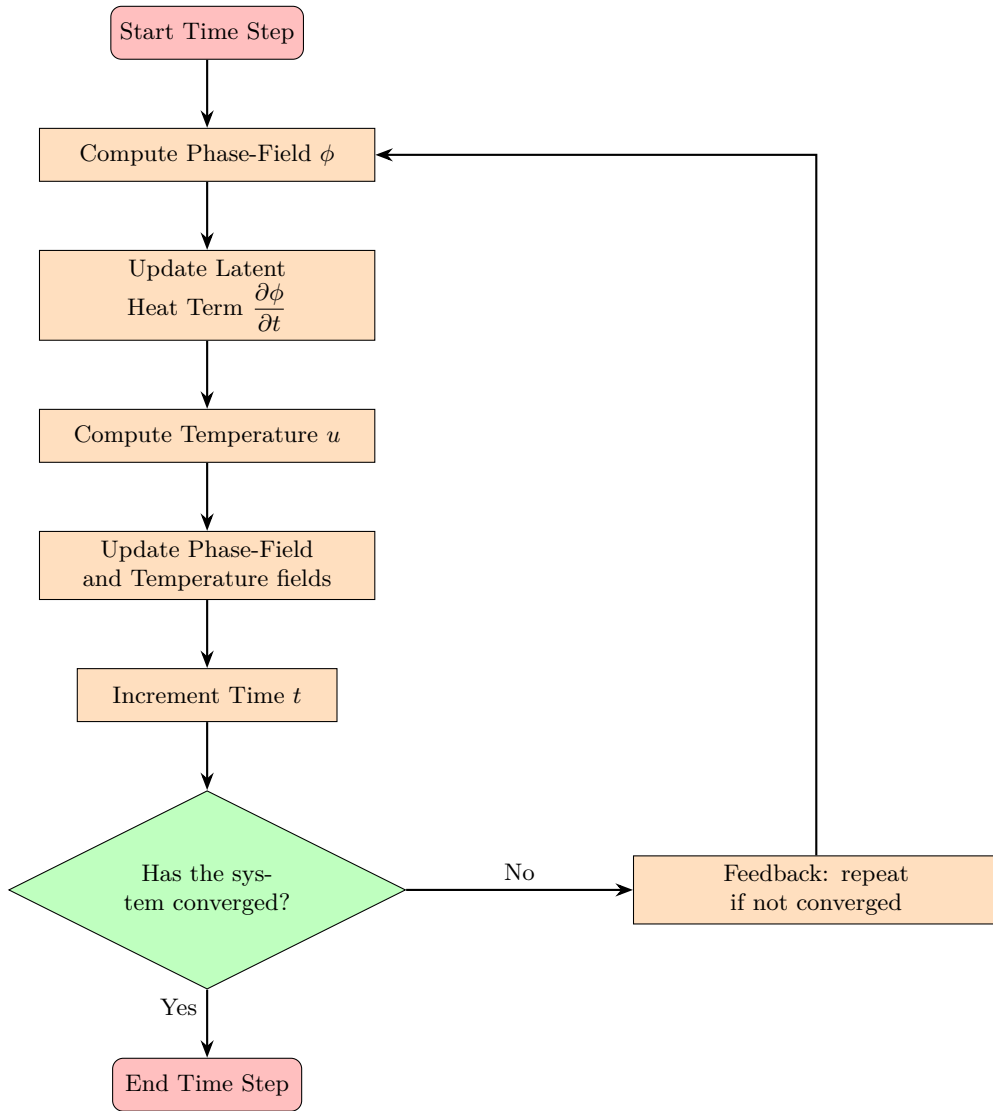


Figure 8: Flow chart of the coupled phase-field (ϕ)-temperature (u) time step in FEniCSx.

The derivative of a_s with respect to n_x is:

$$\frac{\partial a_s}{\partial n_x} = \frac{\partial}{\partial n_x} [(1 - 3\epsilon_4) + 4\epsilon_4 (n_x^4 + n_y^4)]$$

Since the constant term $(1 - 3\epsilon_4)$ does not depend on n_x , its derivative is zero. The derivative of n_x^4 with respect to n_x is:

$$\frac{\partial}{\partial n_x} (n_x^4) = 4\epsilon_4 n_x^3$$

Thus, we have:

$$\frac{\partial a_s}{\partial n_x} = 16\epsilon_4 n_x^3$$

Similarly, the derivative of a with respect to n_y is:

$$\frac{\partial a_s}{\partial n_y} = 16\epsilon_4 n_y^3$$

The derivative of a_s with respect to the gradient $\nabla\phi = (\mathbf{n}_x, \mathbf{n}_y)$ is the vector:

$$\frac{da_s}{d\nabla\phi} = \left(\frac{\partial a_s}{\partial n_x}, \frac{\partial a_s}{\partial n_y} \right)$$

Thus:

$$\frac{da_s}{d\nabla\phi} = (16\epsilon_4 n_x^3, 16\epsilon_4 n_y^3)$$

To ensure that the anisotropy term is normalized, we divide the gradient by its magnitude g_n and project onto the plane perpendicular to the normalized gradient $\mathbf{n}$. The normalized gradient vector is:

$$\mathbf{n} = \frac{(n_x, n_y)}{g_n}$$

We then compute the projection matrix $\mathbf{P}$, which projects any vector onto the plane perpendicular to $\mathbf{n}$:

$$\mathbf{P} = \mathbf{I} - \mathbf{n} \otimes \mathbf{n}$$

where $\mathbf{I}$ is the identity matrix and $\otimes$ denotes the outer product. Finally, we compute the derivative of anisotropy parameter, a_s with respect to the gradient, normalized by g_n :

$$\frac{da}{dg} = \frac{1}{g_n} \cdot \mathbf{P} \cdot (16\epsilon_4 n_x^3, 16\epsilon_4 n_y^3)$$

This is the derivative of the anisotropy term, modulating the interfacial energy according to the direction of the gradient of the phase-field.

The anisotropic flux q_ϕ is given by:

$$\mathbf{q}_\phi = \lambda_0^2 \left(a_s^2 g_\phi + g_2 a_s \frac{da_s}{dg} \right)$$

where λ_0 is a scaling factor, a_s is the anisotropy parameter, g_ϕ is the phase-field gradient, and g_2 is the squared gradient. The term $\frac{da_s}{dg}$ introduces the anisotropic behavior, directing the dendritic growth along the most favorable directions.

The weak form for the anisotropic contribution to the phase-field equation used in the code is :

$$F_{\text{grad}}^{\text{aniso}} = \int_{\Omega} \mathbf{q}_\phi \cdot \nabla w_\phi dx$$

so finally we have the time integrated weak form of the Anisotropy gradient formulation along with temperature,

$$\int_{\Omega} w_\phi \left(\tau_0 \frac{\phi^{n+1} - \phi^n}{\Delta t} + \partial_\phi f(\phi^{n+1}, u^{n+1}) \right) dV + \int_{\Omega} F_{\text{grad}}^{\text{aniso}} dV = 0, \quad (29)$$

$$\int_{\Omega} w_u \left(\frac{u^{n+1} - u^n}{\Delta t} - D \nabla^2 u^{n+1} + \frac{1}{2} \frac{\partial \phi^{n+1}}{\partial t} \right) dV = 0 \quad (30)$$

This term is integrated over the computational domain, contributing to the overall evolution of the phase-field ϕ during the simulation.

The anisotropic term derived here plays a crucial role in directing the dendritic growth along specific directions, enhancing the accuracy of the phase-field model in simulating solidification processes. The directional growth is governed by the four-fold symmetry introduced through the anisotropy function, and the implicit time integration ensures stability over large time steps. In Eq. 30, the coefficient multiplying $\partial_t \phi$ is the (non-dimensional) latent-heat coupling. In the classical symmetric model it is $\frac{1}{2}$ [1], but in our implementation we treat it as a tunable parameter K [2]:

$$\partial_t u = D \nabla^2 u + K \partial_t \phi.$$

Unless otherwise stated we set $K = 0.5$; we also vary K in selected runs to examine its effect on the morphology and tip dynamics.

The simulation, ran on the domain of 300x300, 250x250, 500x500 with parameter provided in the table 3, and some variation of parameter to check how those influence the morphology of the dendrite and also dynamics of the solidification. time step used was $\Delta t = 0.04$ and mesh size $\Delta x = 1$, $\Delta x = 2$ were used. With initial condition given stated in 3.1

- Simulation with domain 300x300, with parameter provided in the table 3 is show in figure 9.
- We tried to vary the parameter and see how the arm develop so we used the parameter with Anisotropy strength, $\epsilon_4 = 0.075$, and the non dimensional latent heat constant $K = 0.75$, to check how the system behaves, the results are almost same but the arm develop a bit early as we can see from the figures at simulation time $t = 150$. The rest of the parameter were same but domain reduced to 250x250 with mesh size $\Delta x = 1$ so as to compensate for the computational cost. The results can be seen in figure 10.

To break perfect symmetry and probe side branching we add a tiny controlled perturbation of the initial phase field (as implemented). To trigger side-branching without breaking symmetry too strongly, we add a tiny, interface-localized Gaussian perturbation to the *initial* phase field only. This practice follows the classic phase-field literature [2]. To visibly resolve side-branching, the diffuse interface must be well resolved; in practice the mesh size should be $h \lesssim \lambda_0/8$ (6-10 elements across the interface) and a time step consistent with diffusion, $\Delta t \lesssim 0.25 h^2/D$.

$$\phi_0^{\text{pert}}(\mathbf{x}) = \text{clip} \left[\phi_0(\mathbf{x}) + \underbrace{5 \times 10^{-4}}_{\text{fixed amplitude}} M(\mathbf{x}) \eta(\mathbf{x}) \right]_{-1}^1, \quad \eta \sim \mathcal{N}(0, 1) \text{ i.i.d.} \quad (31)$$

with the interface mask

$$M(\mathbf{x}) = \text{clip}[1 - \phi_0(\mathbf{x})^2, 0, 1], \quad (32)$$

so bulk regions ($|\phi_0| \approx 1$) remain essentially unperturbed. We use a fixed RNG seed 12345 for reproducibility, and we clip ϕ to $[-1, 1]$ after perturbation.

The code is organized into several blocks in code as shown in pseudo code 11 modular components to ensure readability and maintainability:

- **Mesh Generation:** This module creates the mesh for the computational domain. The mesh is generated using FEniCSx's built-in mesh creation functions with triangular elements. For this problem, a rectangular mesh is created for the square domain, and the mesh is then used for spatial discretizations.
- **Variational Problem Definition:** This module defines the weak formulation of the phase-field equation and anisotropy term. It uses UFL to create variational forms for the temperature field, phase-field, and anisotropy term, and assembles the system of equations.
- **Solution and Time Integration:** This component handles the time-stepping procedure, where the implicit time integration scheme is implemented to solve for the phase-field variable at each time step. The solution of the system is handled using PETSc solvers.
- **Output and Visualization:** The results are saved in XDMF file and visualized using ParaView.

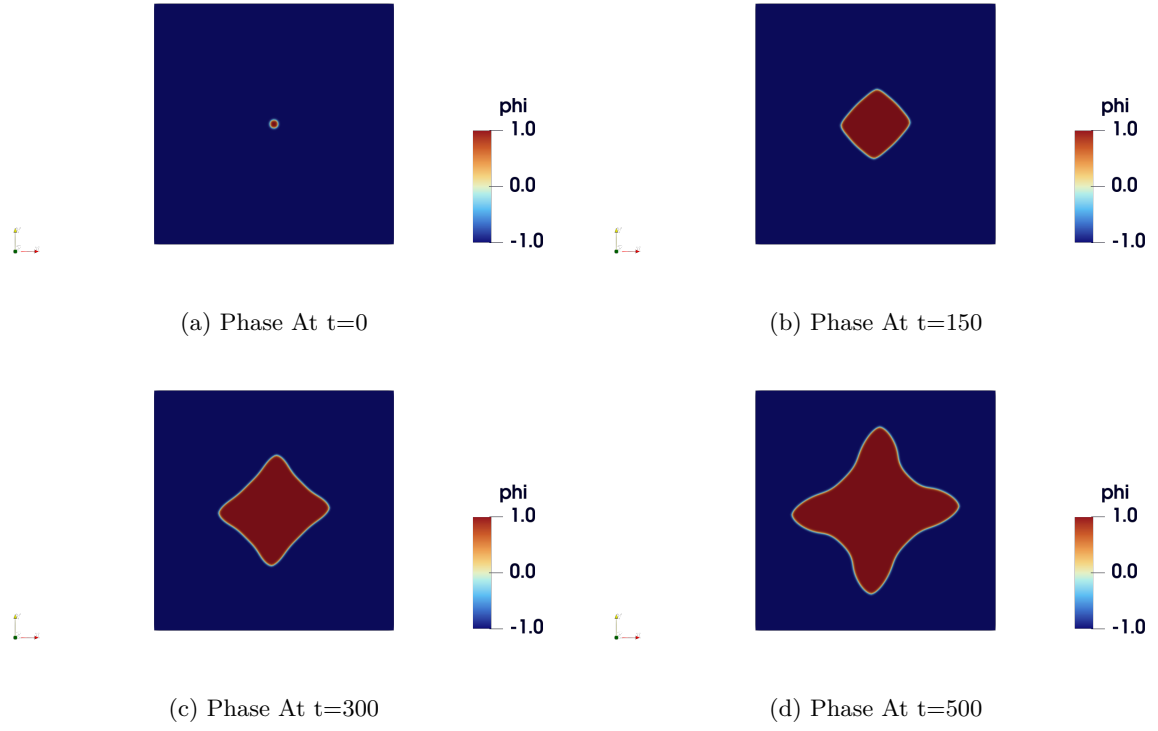


Figure 9: Anisotropic phase evolution with reference paper parameter at different simulation time, t

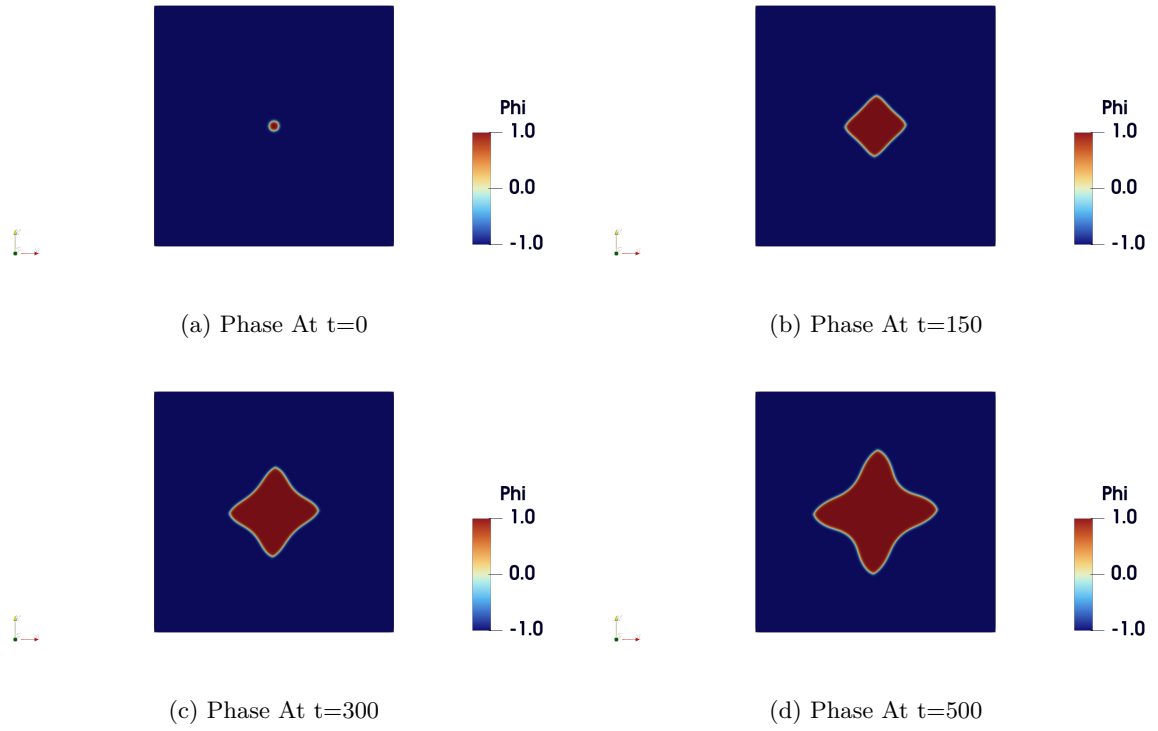


Figure 10: Anisotropic phase evolution with changed parameters $\zeta = 0.075$ and $K = 0.75$ at different simulation time, t

Primary fields and spaces. Solve $(\phi, u) \in H^1(\Omega) \times H^1(\Omega)$ on a 2D triangular mesh $(P^1 \times P^1)$. Time discretization: backward Euler with step Δt .

Parameters (hard-coded). $\zeta=1.6$, $\tau_0=1$, $\lambda_0=1$, $D=1$, $K=0.5$, $\Delta t=0.04$; domain $[0, L_x] \times [0, L_y]$ with $L_x=L_y=250$ and mesh $(N_x, N_y) = (250, 250)$; seed radius $R_0=5$; anisotropy $\epsilon_4=0.05$; regularization $\eta=10^{-8}$; final time $T=500$; write stride **STRIDE**=20.

Free energy derivative (thermal tilt). $f'_\phi(\phi; u) = -\phi + \phi^3 + \zeta u (1 - 2\phi^2 + \phi^4)$.

Anisotropic gradient energy (four-fold) with regularization. Let $\mathbf{g} = \nabla \phi$, $g_2 = \mathbf{g}\mathbf{g}$, $\tilde{g} = \sqrt{g_2 + \eta^2}$, $\mathbf{n} = \mathbf{g}/\tilde{g}$, $\mathbf{I}$ identity, $\mathbf{P} = \mathbf{I} - \mathbf{n} \otimes \mathbf{n}$. $a_s(\mathbf{n}) = (1 - 3\epsilon_4) + 4\epsilon_4(n_x^4 + n_y^4)$, $\frac{\partial a_s}{\partial \mathbf{n}} = (16\epsilon_4 n_x^3, 16\epsilon_4 n_y^3)^\top$, $\frac{\partial a_s}{\partial \mathbf{g}} = \frac{\mathbf{P}}{\tilde{g}} \frac{\partial a_s}{\partial \mathbf{n}}$, $\mathbf{q}_\phi = \lambda_0^2 \left(a_s^2 \mathbf{g} + g_2 a_s \frac{\partial a_s}{\partial \mathbf{g}} \right)$, $\tau = \tau_0 a_s^2$.

Weak forms (monolithic). With tests (w_ϕ, w_u) and previous (ϕ_0, u_0) :

$$\begin{aligned} R_\phi &= \int_{\Omega} \tau(\phi - \phi_0) w_\phi + \Delta t f'_\phi(\phi; u) w_\phi + \Delta t \mathbf{q}_\phi \cdot \nabla w_\phi \, dx, \\ R_u &= \int_{\Omega} (u - u_0) w_u - K(\phi - \phi_0) w_u + \Delta t D \nabla u \cdot \nabla w_u \, dx, \\ R &= R_\phi + R_u, \quad J = \partial R / \partial (\phi, u). \end{aligned}$$

Solver. Implicit, monolithic Newton (incremental; rtol 10^{-8} , atol 10^{-10} , max 100). Linear solve by LU (SuperLU_DIST or MUMPS if available). *If the Newton solve fails to converge at any step, the simulation stops and prints an error.*

Interface-localized noise (at $t=0$ only). $\phi_0 \leftarrow \text{clip}[\phi_0 + 5 \times 10^{-4} (1 - \phi_0^2) \eta]_{-1}^1$, $\eta \sim \mathcal{N}(0, 1)$ (seed 12345).

Output (two files). Write XDMF time series using collapsed subspaces: `out_task3_3/phi_series.xdmf` (for ϕ) and `out_task3_3/u_series.xdmf` (for u), every **STRIDE** steps.

Time-stepping loop (backward Euler; abort on non-convergence)

```

1: Build mesh and  $P^1 \times P^1$  space; create mixed functions com, com_0
2: Set ICs:  $\phi_0(x) = \tanh((R_0 - r)/(\sqrt{2}\lambda_0))$ ,  $u_0(x) \equiv -0.75$ 
3: Apply masked noise to  $\phi_0$ ; set  $(\phi, u) \leftarrow (\phi_0, u_0)$ ;  $t \leftarrow 0$ , step  $\leftarrow 0$ 
4: Open XDMF writers for  $\phi$  and  $u$ ; write  $t=0$  snapshot
5: while  $t < T$  do
6:    $t \leftarrow t + \Delta t$ ; step  $\leftarrow$  step + 1
7:   Assemble  $R, J$  with current  $(\phi_0, u_0)$ 
8:   Newton solve  $R = 0$  for  $(\phi, u)$ 
9:   If not converged: STOP with error message and exit
10:  Update previous:  $(\phi_0, u_0) \leftarrow (\phi, u)$ 
11:  if step mod STRIDE = 0 then
12:    write  $\phi$  and  $u$  at time  $t$ 
13:  end if
14:  if  $|\phi| \approx 1$  at all nodes then
15:    break ▷ early stop: fully solid/liquid
16:  end if
17: end while

```

Figure 11: Pseudo code for the current FEniCSx implementation (anisotropic phase field, temperature, LU solve, abort on non-convergence).

4 Verification

4.1 T1 - Verification of Homogeneous Phase-Field

Aim: Verifying that, with the gradient term removed, the phase-field

- at each node behaves homogeneously over the domain with uniform phase evolution as an independent ODE driven by the bulk energy density and evolve .

- The bulk evolution in the every node matches the analytically calculated bulk function values.

By visualizing the stationary energy landscape $f(\phi, u)$ for $u = -0.75$ and overlaying a representative node's simulated trajectory $(\phi_i(t), F(\phi_i(t), u))$ to confirm downhill motion toward a stable well (tilt set by ζu). Also the simulation result shows homogeneity.

Setup: No-gradient Allen-Cahn kinetics at each DOF:

$$\tau_0 \dot{\phi} = -\frac{\partial f}{\partial \phi}(\phi, u) = \phi - \phi^3 - \zeta u (1 - 2\phi^2 + \phi^4), \quad (33)$$

$$f(\phi, u) = -\frac{1}{2}\phi^2 + \frac{1}{4}\phi^4 + \zeta u \phi \left(1 - \frac{2}{3}\phi^2 + \frac{1}{5}\phi^4\right),$$

Initial condition: We set $\phi(\mathbf{x}, 0) = -0.6$ (constant in Ω) and keep $u(\mathbf{x}, t) \equiv -0.75$ more is explained in 3.2. with fixed $\zeta = 1.6$ and *fixed* u for this verification so that f decreases monotonically in time. We use an interior node (e.g. node #100) that undergoes a clean liquid→solid transition.

How to run Using the plotting script, `Testfvsphi.py`, which uses the data stored in excel file of the simulation `task_0.py`, so they need to be kept in the same folder, The current script loads columns `phi_node` and `f_node`, evaluates $f(\phi, u)$ on a grid for u , and overlays the node trajectory.

```
1 # in terminal or IDEs
2
3 python Testfvsphi.py
```

Expected result:

- **Energy landscape:** For fixed u , the curve $f(\phi, u)$ shows a tilted double well $\text{sign}(\zeta u)$ determines the favored well.
- **Trajectory consistency:** The node's $(\phi_i(t), f(\phi_i(t), u))$ points *lie on* the corresponding analytical $f(\phi, u)$ curve and move *downhill in* f over time toward the favored minimum.
- **Visual:** The domain should homogeneously evolve.

Obtained result

- Figure 12 shows the double well bulk curve for u value with the simulation trajectory of node #choice = 100 overlaid. The trajectory adheres to the appropriate analytical $f(\phi)$ curve and drifts monotonically toward the favored well consistent with $\text{sign}(\zeta u)$.
- In figure 13 we can clearly see the domain uniformly evolving.

Figure 12 overlays the simulated node trajectory $(\phi_i(t), f(\phi_i(t), u))$ on top of the stationary bulk energy curve $F(\phi) = f(\phi, u)$ for $u = -0.75$ and $\zeta = 1.6$. The following observations support correctness of the no-gradient kinetics:

- **On-curve consistency.** Every simulated point $(\phi_i(t_k), f_i(t_k))$ lies on the analytical $F(\phi)$ curve for the same u within machine precision, confirming that $f_i(t)$ was computed from $\phi_i(t)$ using the correct bulk functional.
- **Downhill motion toward the favored well.** For $\zeta u < 0$ the negative- ϕ well is energetically disfavored; the trajectory advances along $F(\phi)$ toward the positive- ϕ minimum, as seen by the arrowed path in the lower panel. This matches the expected tilt direction set by $\text{sign}(\zeta u)$.
- Homogeneity and function space is correctly assigned and no spillage and blowup of values at each node.

Conclusion. The overlay confirms that the no-gradient phase-field dynamics reduce to the intended local ODE at each node and dissipate the bulk free energy for fixed u . All acceptance criteria are met on this test case, providing a clear verification of the code.

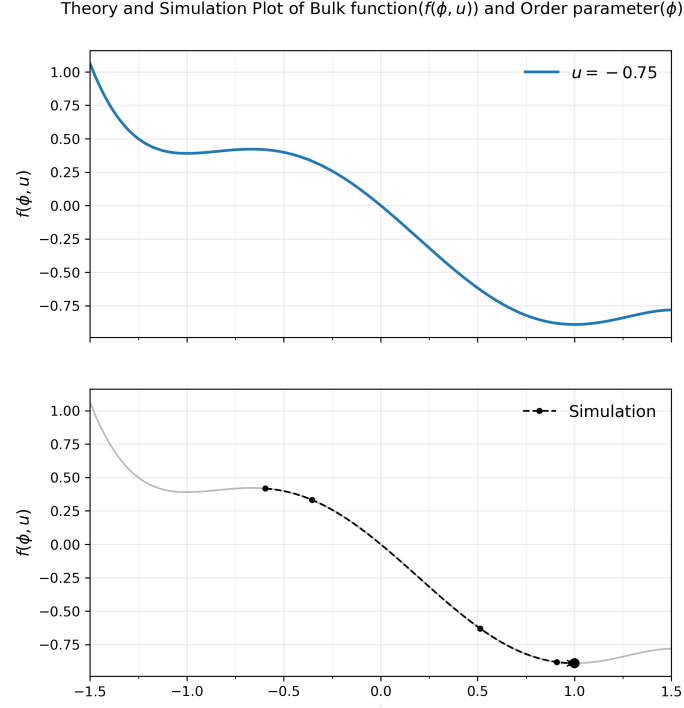


Figure 12: Comparison plot between analytically obtained $f(\phi, u)$ vs ϕ at constant u and simulation.

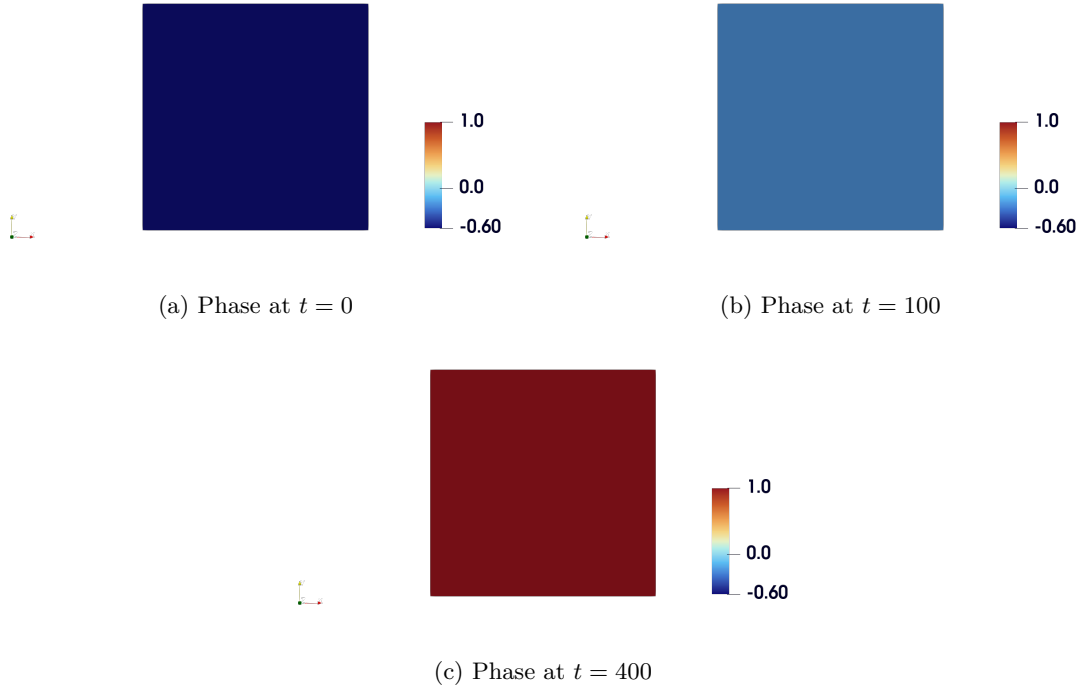


Figure 13: Homogeneous phase evolution at different time steps. The domain remains uniform in the no-gradient test.

4.2 T2 - Gradient term & Interface tracking (flat front)

Aim: Verify that enabling the gradient penalty produces a diffuse interface with physically consistent behavior: (i) monotone decay of the *total* free energy, (ii) constant-speed motion of a flat front under a uniform thermodynamic tilt, (iii) expected parameter scaling of the speed with λ_0 and τ_0 , and (iv) mesh/time-step independence of the measured speed and interface thickness.

Setup: Allen-Cahn kinetics with gradient penalty:

$$\tau_0 \dot{\phi} = -\mu, \quad \mu = \frac{\partial f}{\partial \phi}(\phi, u) - \lambda_0^2 \nabla^2 \phi, \quad (34)$$

with bulk energy density

$$f(\phi, u) = -\frac{1}{2}\phi^2 + \frac{1}{4}\phi^4 + \zeta u \phi \left(1 - \frac{2}{3}\phi^2 + \frac{1}{5}\phi^4\right). \quad (35)$$

Total (Lyapunov) free energy in Ω :

$$\Pi[\phi] = \int_{\Omega} \left(f(\phi, u) + \frac{\lambda_0^2}{2} |\nabla \phi|^2 \right) d\Omega. \quad (36)$$

For fixed u and standard no-flux/periodic BC, $\frac{d}{dt}\Pi \leq 0$.

Flat interface test in a rectangular domain:

- *Domain/mesh:* $L_x \times L_y$ rectangle (e.g. 50×50), structured mesh with mesh size $\Delta x = 1$.
- *Boundary conditions:* periodic or homogeneous Neumann on all sides.
- *Initial condition (partitioned):* half-solid / half-liquid vertical step

$$\phi(x, 0) = \begin{cases} +1, & x < L_x/2, \\ -1, & x \geq L_x/2, \end{cases}$$

creating a single flat interface at $x \approx L_x/2$.

- *Parameters (baseline):* $\zeta = 1.6$, $u = -0.75$, $\tau_0 = 1.0$, $\lambda_0 = 1.0$, $\Delta t = 0.04$.
- *Interface tracking:* front position $x^*(t)$ from the solid area $A_s(t) = \int_{\Omega} \frac{\phi+1}{2} d\Omega$; for a vertical front $A_s(t) = (L_x - x^*(t))L_y$ hence $x^*(t) = L_x - A_s(t)/L_y$.

Code artefacts. Main solver: `task_1.py` (Task-1 implementation). Test harness with partitioned IC & trackers: `task_0_2.py`. Post-processor for interface lines: `interface_track.py`. (Only the *initial condition* differs; PDE, parameters, time stepping and FE spaces are identical.)

How to run.

```

1
2 # 1) Baseline run (writes interface_track.csv; energy_track.csv and
   ↳ thickness_track.csv).
3 python task_0_2.py
4
5 # 2) Plot graphs and summary of post process
6 python interface_track.py

```

Expected result.

- *Energy decay:* $\Pi(t)$ in (36) should be decreasing as solid front moves.
- *Flat-front kinematics:* $x^*(t)$ is linear in t while the front is far from boundaries $\Rightarrow$ constant speed v ; R^2 of the linear fit ≈ 1 .
- *Constant velocity :* keeping u, ζ fixed, measured v remains constant once get stabilized. and decreases roughly inversely with τ_0 (empirical trend $v \propto \lambda_0/\tau_0$).

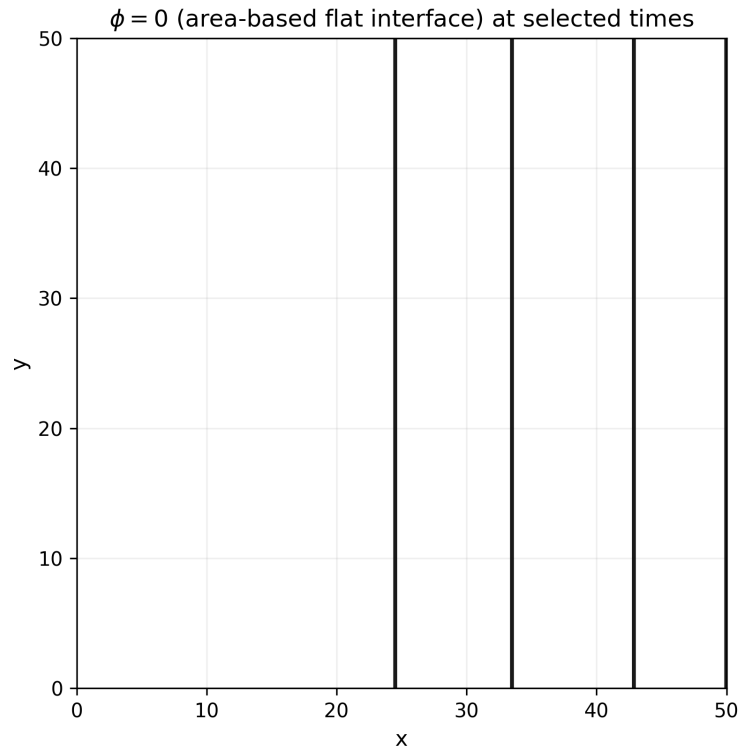


Figure 14: Tracked interface positions $x^*(t_k)$ (vertical lines) at uniform time intervals; equal spacing indicates constant speed.

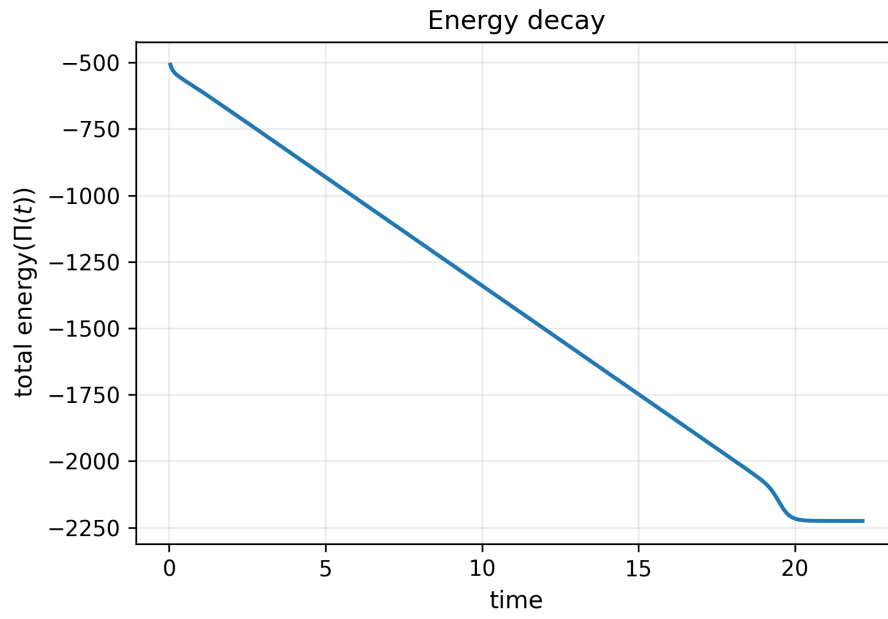


Figure 15: Total energy vs. time; monotone decay confirms dissipation with the gradient term.

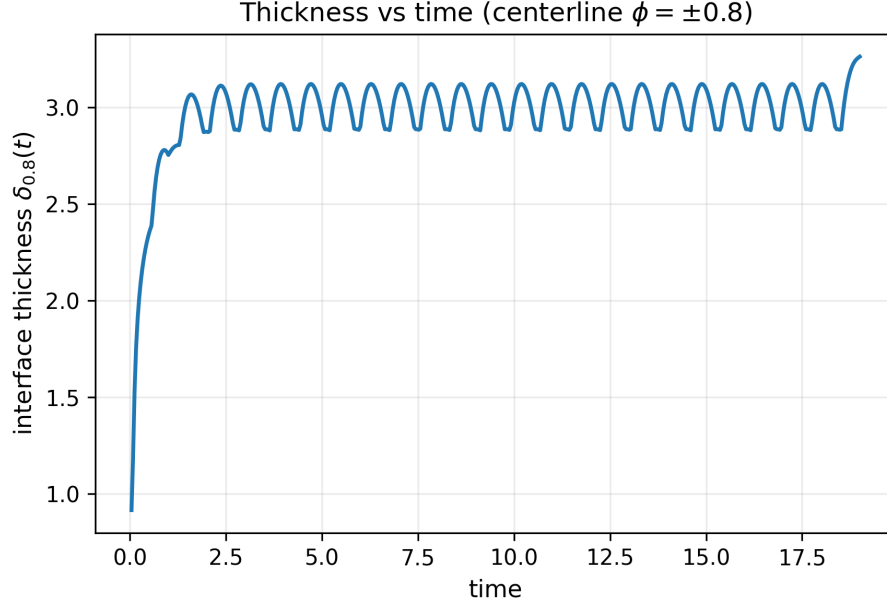


Figure 16: Interface thickness variation with simulation time.

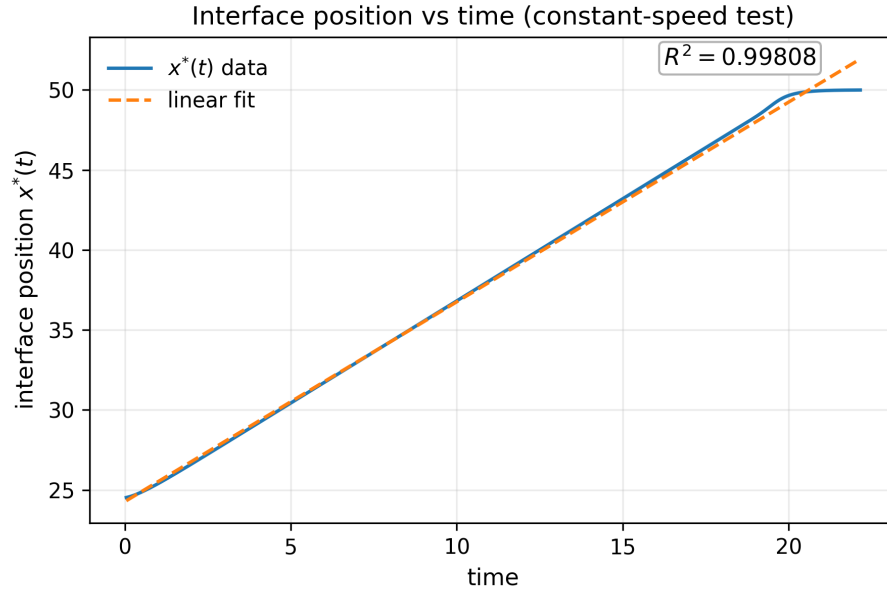


Figure 17

Table 4: Quantitative checks (baseline: $\lambda_0 = 1$, $\tau_0 = 1$, $\zeta = 1.6$, $u = -0.75$).

Metric	Value	Verdict
constant speed v	1.2483	Pass (from fit)
Linear fit coefficient of determination R^2	0.998082	Pass (≥ 0.998)
Max relative non-monotonicity in $\mathcal{E}$	≈ 0	Pass ($< 10^{-6}$)
Thickness $\delta_{0.8}$ (median)	3.020	Report (scales with λ_0)

Notes: Speed and R^2 from the interior-window fit of $x^*(t)$; energy shows no up-ticks within plotting precision; the final thickness becomes undefined when the front touches the boundary, so the *median* value (3.020) is reported.

Table 5: Analytical vs Numerical comparison

Quantity	Theory	Numerical	Abs. error	Rel. error
Interface speed $ v $	1.3576	1.2483	0.1093	8.05%
Thickness $\delta_{0.8}$	3.1090	3.0200	0.0890	2.86%

Notes: The sign of v depends on orientation; the table reports magnitudes. The thickness numerical value is the median over the interior time window (front far from boundaries).

Obtained result:

Thickness vs time. From the figure 16 The interface thickness $\delta_{0.8}(t)$ quickly settles to a constant value set by λ_0 (i.e. a stable diffuse interface). The small near-sinusoidal wiggle comes from how thickness is measured on the grid: we detect the $\phi = \pm 0.8$ crossings along one row and linearly interpolate between cell centers; as the front translates, the threshold slides from one cell to the next, producing a mild, periodic measurement ripple. The envelope is flat, which is the key. Near the wall the centerline thickness estimate becomes noisy so the ripple amplitude grows. This can be minimized by decreasing the mesh size, as the distance between the grids becomes small the wiggle also decreases.

Interface position vs time. From the plot in figure 17 we see that the front position $x^*(t)$ lies on a straight line with $R^2 \approx 1$, showing constant-speed motion under a uniform driving u . The slope of the fit is the interface speed.

Energy decay. From the curve in the figure 15 we can say that, the total free energy $\Pi(t) = \int_{\Omega} (f(\phi, u) + \frac{\lambda_0^2}{2} |\nabla \phi|^2) d\Omega$ decreases monotonically, as expected for an Allen–Cahn gradient flow. No meaningful upward bumps are observed, confirming the gradient term has the correct sign and the time integration is stable. The curve levels off when the system is fully solid; no more interface so no more dissipation.

Notes:

- The gradient term contributes only where $\nabla \phi \neq 0$; a homogeneous IC would not exercise it.
- The harness `task_0_2.py` differs from `task_1.py` only by the IC and diagnostics (interface/energy logging); both solve the same PDE with the same FEM and parameters.
- Keep the front far from boundaries during measurement; discard initial transients and late-time boundary interactions when fitting $x^*(t)$.

Energy dissipation law

With Allen–Cahn kinetics and a gradient penalty,

$$\tau_0 \dot{\phi} = -\mu, \quad \mu = \frac{\partial f}{\partial \phi}(\phi, u) - \lambda^2 \nabla^2 \phi,$$

and the total free energy (Plapp–Karma form),

$$\Pi[\phi, u] = \int_{\Omega} \left(f(\phi, u) + \frac{\lambda^2}{2} |\nabla \phi|^2 \right) d\Omega, \quad f(\phi, u) = -\frac{1}{2} \phi^2 + \frac{1}{4} \phi^4 + \xi u \phi \left(1 - \frac{2}{3} \phi^2 + \frac{1}{5} \phi^4 \right),$$

one obtains, for fixed u and periodic or no-flux (Neumann) boundaries, the Lyapunov (energy-dissipation) identity:

$$\frac{d\Pi}{dt} = \int_{\Omega} \left(\frac{\partial f}{\partial \phi} \dot{\phi} + \lambda^2 \nabla \phi \cdot \nabla \dot{\phi} \right) d\Omega = \int_{\Omega} \mu \dot{\phi} d\Omega = -\frac{1}{\tau_0} \int_{\Omega} \mu^2 d\Omega \leq 0.$$

Thus $\Pi[\phi, u]$ decreases monotonically in time, as observed in our simulations [6].

Hence *the correct sign and coefficient of the Laplacian term in μ imply that the total energy is strictly non-increasing in time.* In discrete time (e.g., backward Euler), a fully converged step inherits this decay up to solver/tolerance effects; small upward blips indicate either too large Δt or insufficient nonlinear/linear solve accuracy.

Analytic reference values for thickness and speed

We use the Plapp–Karma bulk density [3]

$$f(\phi, u) = -\frac{1}{2}\phi^2 + \frac{1}{4}\phi^4 + \xi u \phi \left(1 - \frac{2}{3}\phi^2 + \frac{1}{5}\phi^4\right), \quad (37)$$

and the Allen–Cahn kinetics with gradient penalty

$$\tau_0 \dot{\phi} = -\mu, \quad \mu = \frac{\partial f}{\partial \phi}(\phi, u) - \lambda^2 \nabla^2 \phi. \quad (38)$$

(i) Interface thickness at small tilt: For $u = 0$ the stationary 1D profile is the classic tanh-kink

$$\phi_0(x) = \tanh\left(\frac{x}{\sqrt{2}\lambda}\right), \quad (39)$$

so the thickness measured between the $\phi = \pm 0.8$ crossings is

$$\delta_{0.8}^{(\text{theory})} = 2\sqrt{2}\lambda \operatorname{atanh}(0.8) = 2\sqrt{2}\lambda \times 1.098612 \approx 3.109. \quad (40)$$

With $\lambda_0 = 1$ this gives $\delta_{0.8}^{(\text{theory})} \approx 3.109$. In our run (finite tilt $u = -0.75$) the measured median was $\delta_{0.8}^{(\text{median})} \approx 3.020$, i.e. an error of $\approx (3.109 - 3.020)/3.109 \simeq 2.9\%$, consistent with weak bias and discrete sampling.

(ii) Constant-speed of a flat front under a uniform bias: Let Δf be the free-energy difference between the wells at $\phi = \pm 1$: from (37),

$$f(\pm 1, u) = -\frac{1}{4} \pm \xi u \frac{8}{15} \quad \Rightarrow \quad \Delta f \equiv f(+1, u) - f(-1, u) = \frac{16}{15} \xi u.$$

For small uniform driving Δf , the Allen–Cahn traveling-wave analysis gives [5, 6]

$$v = \frac{\Delta f}{\tau_0 \int_{-\infty}^{\infty} (\phi'_0)^2 dx} = \frac{\Delta f}{\tau_0} \frac{3\sqrt{2}\lambda}{4} = \frac{4}{5} \frac{\sqrt{2}\lambda}{\tau_0} (\xi u), \quad (41)$$

where we used ϕ_0 from (39) and $\int_{-\infty}^{\infty} (\phi'_0)^2 dx = 4/(3\sqrt{2}\lambda)$. With $\lambda = \tau_0 = 1$ and $\xi u = 1.6 \times (-0.75) = -1.2$,

$$|v|_{\text{theory}} = \frac{4}{5} \sqrt{2} |\xi u| \approx 1.13137 \times 1.2 \approx 1.358.$$

The linear fit of $x^*(t)$ over the interior time window gives $v_{\text{fit}} \approx 1.248$ with $R^2 \simeq 0.998$, i.e. an $\approx 8.1\%$ deficit, which is expected for a non-infinitesimal bias, finite mesh/time step, and the area-based definition of $x^*(t)$.

References: The form of $f(\phi, u)$ and the tanh-profile/interface-energy machinery follow the Plapp–Karma phase-field model [3]. The speed formula (41) is the standard Allen–Cahn traveling-wave result for weak uniform driving (energy difference Δf) evaluated using the tanh profile [5, 6].

4.3 T3 - Verification of Temperature and Phase-Field Coupling

Model–code alignment. The solver advances the mixed field (ϕ, u) with a monolithic, fully implicit Backward–Euler step on a $P_1 \times P_1$ space. The assembled residuals in `task_2.py` are

$$R_u = \int_{\Omega} \left[(u - u_0) - \frac{1}{2}(\phi - \phi_0) \right] w_u dx + \Delta t D \int_{\Omega} \nabla u \cdot \nabla w_u dx \quad (42)$$

$$R_{\phi} = \int_{\Omega} \frac{\phi - \phi_0}{\Delta t} w_{\phi} dx + \int_{\Omega} \left(-\phi + \phi^3 + \zeta u (1 - 2\phi^2 + \phi^4) \right) w_{\phi} dx + \lambda_0^2 \int_{\Omega} \nabla \phi \cdot \nabla w_{\phi} dx, \quad (43)$$

with homogeneous Neumann (zero-flux) BCs for both fields.

Numerics present in the code.

- **Diagnostics.** `test2.py` defines `CoupledDiagnostics` class which logs: total free energy $\Pi(t)$, a non-positive dissipation proxy $H(t)$, the integral balance $\Delta f u$ vs. $\frac{1}{2} \Delta f \phi$, and their misfit. Artifacts: `diag_out/diagnostics.csv` and PNG plots.

- **BC check.** `test_bc.py` checks the net fluxes $\int_{\partial\Omega} \nabla\phi \cdot n \, ds$ and $\int_{\partial\Omega} \nabla u \cdot n \, ds$ (PASS/FAIL), and also reports L^1/L^2 residuals on $\partial\Omega$.

How To run: The two test code run when we use it as import in the testing code.

```

1      # place this in the import section of the code which is to be
      ↪ tested
2  from test2 import CoupledDiagnostics
3  from test_bc import bc_check
4
5  # calling the function inside the test2 file
6  diag = CoupledDiagnostics(msh, phi, u, lamda_0, zet, tau=tau_0,
      ↪                          D=D, plot_every=10)
7  diag.start()
8
9  # in the time loop after solve()
10 diag.update(t) #updates every time step and records parameters
11
12 # after loop at end
13 diag.finish()
14
15 # Now calling the boundary condition test, this can also be called
      ↪ inside time loop
16
17 bc_check(msh, phi_fun, u_fun, tol=1e-8)

```

T1 — Enthalpy–coupling identity

Aim: Verify the enthalpy coupling in the temperature residual by tracking

$$\Delta \int_{\Omega} u \, dx \quad \text{vs.} \quad \frac{1}{2} \Delta \int_{\Omega} \phi \, dx.$$

How it is implemented: At each step, `CoupledDiagnostics.update` assembles the domain integrals of u and ϕ and records the misfit $\Delta \int u - \frac{1}{2} \Delta \int \phi$.

What this plot verifies: With homogeneous Neumann BCs, integrating the coded heat equation over Ω and using $\int_{\partial\Omega} \nabla u \cdot n \, ds = 0$ gives the enthalpy identity

$$\frac{d}{dt} \int_{\Omega} u \, dx = \frac{1}{2} \frac{d}{dt} \int_{\Omega} \phi \, dx \implies \Delta \int_{\Omega} u \, dx = \frac{1}{2} \Delta \int_{\Omega} \phi \, dx.$$

Therefore, plotting $\Delta \int u$ and $0.5 \Delta \int \phi$ on the same axes tests *exactly* the correctness of the temperature–phase-field coupling in the code: the factor $\frac{1}{2}$, its *sign*, its placement in the temperature residual, and the time discretization/assembly. Any mistake (missing $1/2$), wrong sign, lost term, or a BC bug) causes a visible drift between the two curves. **Metric and pass criterion used.** We define the misfit

$$m(t) = \Delta \int_{\Omega} u \, dx - \frac{1}{2} \Delta \int_{\Omega} \phi \, dx,$$

$$r_{\max} = \frac{\max_{0 \leq t \leq T} |m(t)|}{\max \left\{ \max_{0 \leq t \leq T} \left| \Delta \int_{\Omega} u \, dx \right|, \max_{0 \leq t \leq T} \left| \frac{1}{2} \Delta \int_{\Omega} \phi \, dx \right| \right\}}.$$

Criterion: *PASS* if $r_{\max} \leq 10^{-3}$; otherwise *FAIL*. Small oscillations in $m(t)$ are acceptable and reflect solver tolerances.

Expected: The two curves are coincident within plotting resolution; $m(t)$ stays near zero and $r_{\max} \leq 10^{-3}$. This confirms:

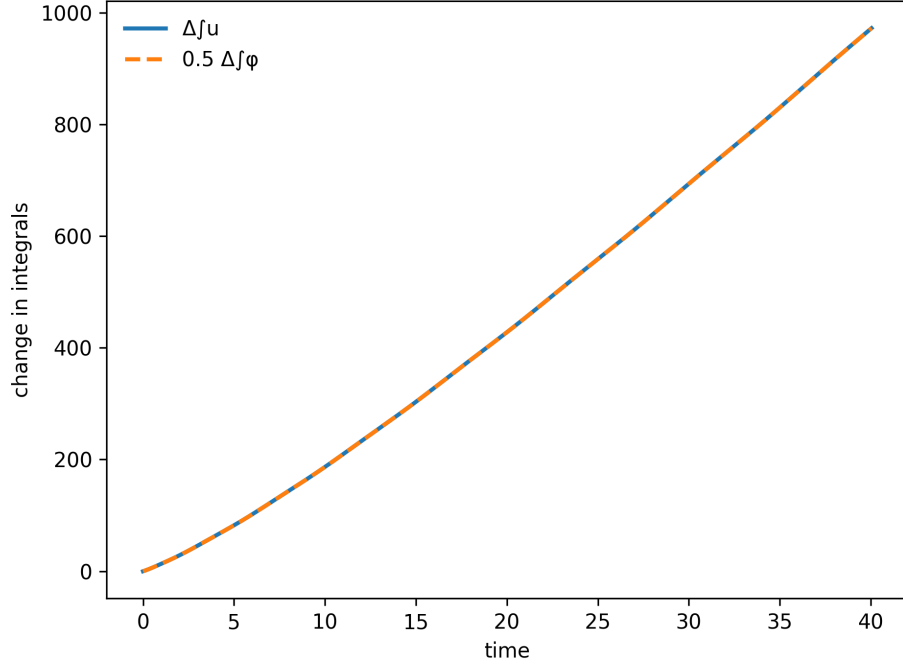


Figure 18: Integral balance from CoupledDiagnostics: $\Delta f u$ vs. $0.5 \Delta f \phi$.

- the $\frac{1}{2} \partial_t \phi$ source is present with the correct sign and scaling
- The Backward-Euler assembly is consistent (no missing Δt factors)
- boundary fluxes are effectively zero so no extra boundary term contaminates the balance.

Observed (from run artifacts): The curves overlap throughout $t \in [0, T]$ and the misfit remains visually flat (cf. Fig. 18); numerically, $r_{\max}$ is within the tolerance (see `diag_out/diagnostics.csv`). . The output from the console after end of simulation is,

```
1 [verify] max|misfit| = 3.231e-09 ; r_max = 3.774e-12 -> PASS
2
```

Code Listing 1: Misfit output

If this test fails, typical causes. Wrong sign or missing $\frac{1}{2}(K)$ in the temperature residual; using ϕ instead of $(\phi - \phi_0)$ in the discrete update; a stray Dirichlet/Robin BC or nonzero normal flux; inconsistent units/scaling of u vs ϕ ; or an assembly bug that drops the coupling term from the Jacobian (Newton divergence or large $m(t)$ drift).

T2 — Energy and dissipation monitors

Aim: Check qualitative thermodynamic consistency of the coupled evolution.

How it is implemented: CoupledDiagnostics computes the free energy

$$\Pi(t) = \int_{\Omega} \left(\frac{\lambda_0^2}{2} |\nabla \phi|^2 + W(\phi, u) \right) dx,$$

and a dissipation/production proxy

$$H(t) = - \int_{\Omega} \left(\lambda_0^2 |\nabla \phi|^2 + D |\nabla u|^2 \right) dx \leq 0,$$

which is insensitive to the detailed algebra of $W(\phi, u)$ and therefore a robust sign check for diffusion/interfacial terms and BCs.

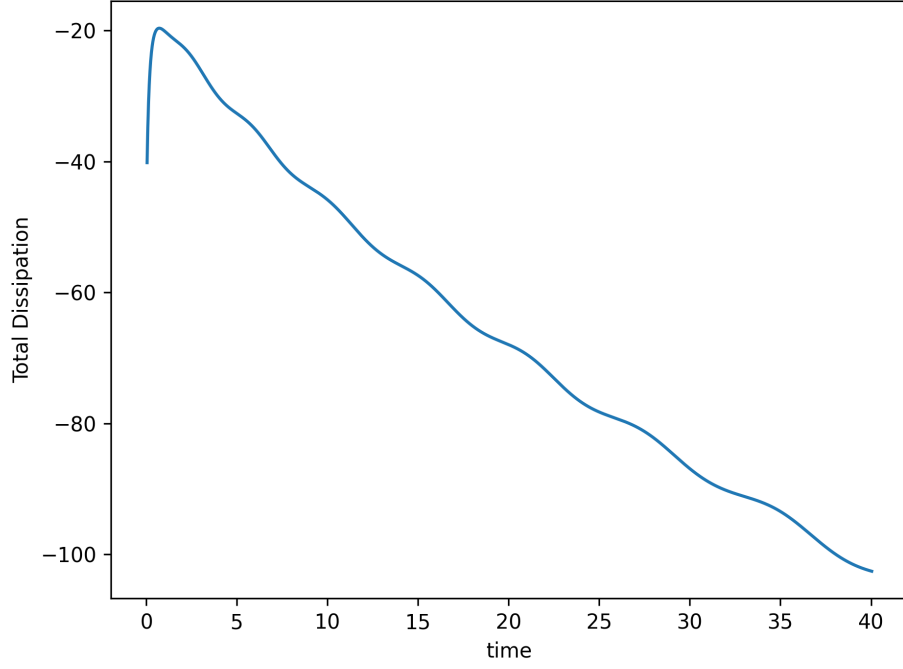


Figure 19: Dissipation proxy $H(t) \leq 0$ from CoupledDiagnostics.

Observed (from run artifacts): $\Pi(t)$ decreases monotonically, and $H(t)$ remains negative with moderate magnitude, indicating smoothing by diffusion and by the interfacial penalty—consistent with the coded model.

Why this helps. If any sign mistake is introduced in $D\nabla^2 u$, in the $\frac{1}{2}\partial_t \phi$ source of the heat equation, or in the boundary terms, $H(t)$ typically flips sign or shows nonphysical growth. Seeing $H(t) \leq 0$ together with a decreasing $\Pi(t)$ is a compact physics check that complements the algebraic balance in Fig. 18.

T3 — Boundary conditions: homogeneous Neumann

Aim. Confirm that zero-flux BCs are satisfied globally at the end of the run.

How it is implemented: `bc_check` function in a test code `test_bc.py` assembles

$$\int_{\partial\Omega} \nabla\phi \cdot \mathbf{n} \, ds, \quad \int_{\partial\Omega} \nabla u \cdot \mathbf{n} \, ds,$$

and also reports L^1 and L^2 residuals of the normal flux on $\partial\Omega$. A PASS is declared when both net fluxes are below the user tolerance (default 10^{-8}). Homogeneous Neumann conditions are natural in the weak form and are satisfied exactly only at the continuous level. In the discrete problem we monitor the normal fluxes on the boundary, $\|\mathbf{n} \cdot \nabla\phi\|_{\infty, \partial\Omega}$ and $\|\mathbf{n} \cdot \nabla u\|_{\infty, \partial\Omega}$. When the diffuse interface approaches $\partial\Omega$ the measured values are typically in the range 10^{-6} – 10^{-7} (max-norm), which is consistent with weak enforcement, mesh resolution, and solver round off. We therefore deem the BCs satisfied provided the boundary residual remains below a practical tolerance, e.g. $\varepsilon_{\text{BC}} = 10^{-6}$ in max-norm (10^{-8} in L^2 is also reasonable). This was only seen in the temperature BC.

Obtained (from console):

```

1 [BC test] Checking homogeneous Neumann (zero-flux) BCs
2 [BC test] Net flux:  $\int_{\partial\Omega} \nabla\phi \cdot \mathbf{n} \, ds = +0.000\text{e}+00$ ,  $\int_{\partial\Omega} \nabla u \cdot \mathbf{n} \, ds = +0.000\text{e}+00$ 
3 [BC test] L1-resid:  $\int_{\partial\Omega} |\nabla\phi \cdot \mathbf{n}| \, ds \approx 0.000\text{e}+00$ ,  $\int_{\partial\Omega} |\nabla u \cdot \mathbf{n}| \, ds \approx 0.000\text{e}+00$ 
4 [BC test] L2-resid:  $(\int_{\partial\Omega} (\nabla\phi \cdot \mathbf{n})^2 \, ds)^{\frac{1}{2}} = 0.000\text{e}+00$ ,  $(\int_{\partial\Omega} (\nabla u \cdot \mathbf{n})^2 \, ds)^{\frac{1}{2}} = 0.000\text{e}+00$ 
5 [BC test] PASSED (net-flux tol=1e-08)

```

Code Listing 2: BC test output

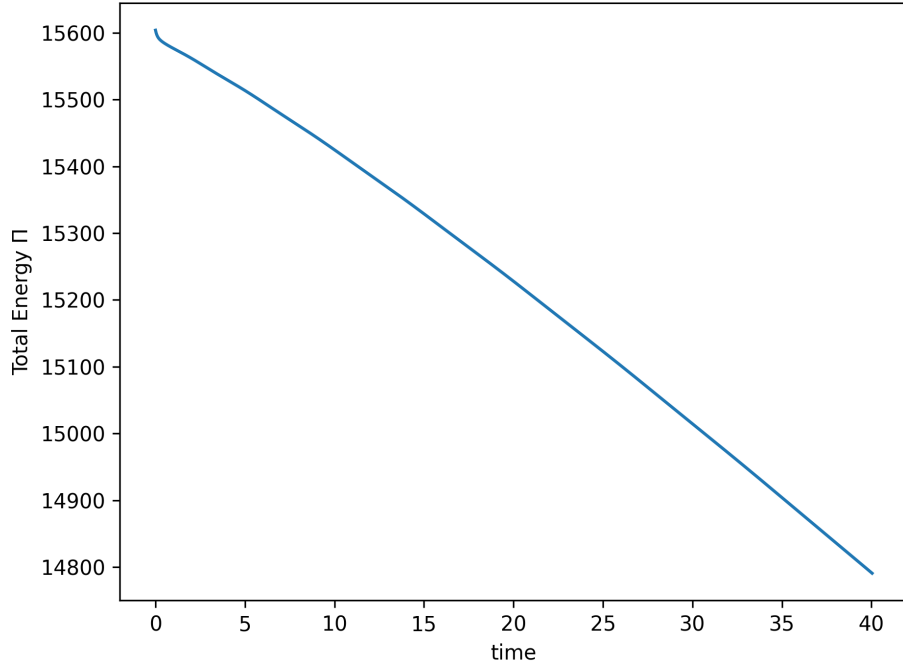


Figure 20: Free energy $\Pi(t)$ from CoupledDiagnostics.

Reproducibility (what the code already does). Running `python task_2.py` generates XDMF time series for ϕ and u and writes diagnostics under `diag_out/`: `diagnostics.csv`, `balance.png`, `Energy_vs_time.png`, `Dissipation_vs_time.png`, `misfit.png`. The BC checker runs after the time loop and reports PASS/FAIL in the terminal.

4.4 T4 - Decoupling Tests

T1 - Diffusion of u with ϕ frozen

Aim: Verifies that the temperature field u diffuses under homogeneous Neumann BCs while the phase field remains fixed ($\phi \equiv -1$). This checks the diffusion operator and decoupling for $K = \zeta = 0$.

Setup: The evolution equation is

$$u_t = D \nabla^2 u \quad \text{in } \Omega = [0, 1]^2, \quad \nabla u \cdot n = 0 \text{ on } \partial\Omega,$$

discretized by P_1 FE in space and fully implicit in time. The phase field is held at $\phi \equiv -1$ (no motion). Initial condition for u is the (1, 1) Neumann eigenmode:

$$u(x, y, 0) = \cos(\pi x) \cos(\pi y).$$

How to run:

```
1 python Test_f1.py
2
```

This simulation uses the initializers given in the parameters in Table 6.

Expected result: We expect the phase to not evolve during the simulation and remain same as initialized, where as u changes with time independent of ϕ .

Obtained result (visual): From the results obtained, we can see that in figures 21 and 22 the phase does not evolve but the temperature seems to have small diffusion.

Conclusion: The diffusion operator with Neumann BCs and the BE integrator behave as expected; decoupling holds (ϕ frozen).

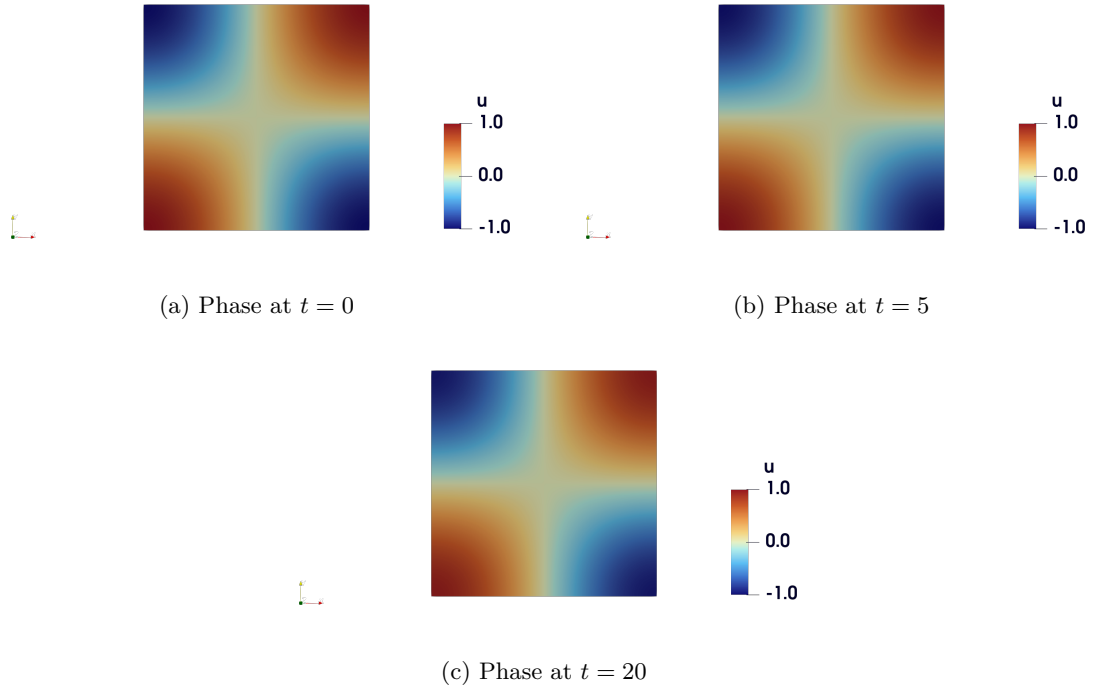


Figure 21: The evolution of temperature in a decoupled system at different simulation time, t

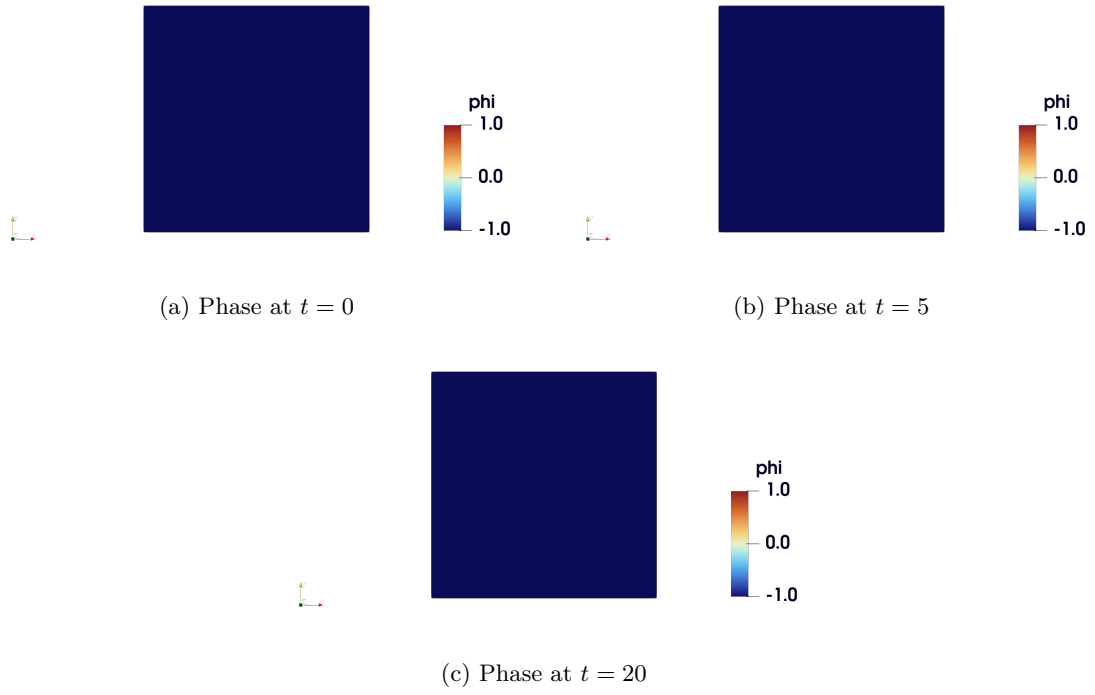


Figure 22: The evolution of phase in a decoupled system at different simulation time, t

Table 6: Parameters for Test A (u-only diffusion).

Quantity	Symbol	Value
Domain size	$L_x = L_y$	100
Mesh cells	$N_x = N_y$	100
Time step	dt	0.05
Final time	T	1.0
Write stride		5
Diffusivity	D	1.0
Latent-heat coupling	K	0.0
Bulk coupling	ζ	0.0
Anisotropy	ε_{an}	0.0
Interface width	λ_0	1.0
Kinetic prefactor	τ_0	1.0
Initial mean of u	u_0	0.0
Phase field	$\phi(x, 0)$	-1 (constant)
Temperature IC	$u(x, 0)$	$\cos(\pi x) \cos(\pi y)$

T 2 - Phase-field-only evolution (central seed)

Aim: Verify that the phase field ϕ evolves correctly when decoupled from u : a circular seed $\phi \approx +1$ is embedded in $\phi \approx -1$, shrinks under curvature; u stays constant (no diffusion, no latent heat).

Setup: We solve the ϕ equation (double-well bulk + isotropic gradient penalty) with $K = 0$, $\zeta = 0$, $D = 0$. Initial condition:

$$\phi(x, y, 0) = \tanh\left(\frac{r_0 - r}{w_{\text{eq}}}\right), \quad r = \sqrt{(x - L_x/2)^2 + (y - L_y/2)^2}, \quad w_{\text{eq}} = \sqrt{2} \lambda_0,$$

with a central seed of radius r_0 . Expect curvature-driven collapse and the u field remain constant in time.

How to run:

1
2

```
python Test_f2.py
```

Parameters are shown in the Table 7.

Table 7: Parameters for Test B (ϕ -only, circular seed).

Quantity	Symbol	Value
Domain size	$L_x = L_y$	100.0
Mesh cells	$N_x = N_y$	100
Time step	dt	0.01
Final time	T	20.0
Write stride		5
Diffusivity	D	0.0
Latent-heat coupling	K	0.0
Bulk coupling	ζ	0.0
Anisotropy	ε_{an}	0.0
Interface width	λ_0	1.0
Kinetic prefactor	τ_0	1.0
Seed radius	r_0	3.0
Temperature IC	$u(x, 0)$	0 (constant)
Phase IC	$\phi(x, 0)$	$\tanh((r_0 - r)/(\sqrt{2}\lambda_0))$

Expected result: ϕ forms a clear +1 island that *shrinks* and vanishes. The u field remains exactly constant in space and time.

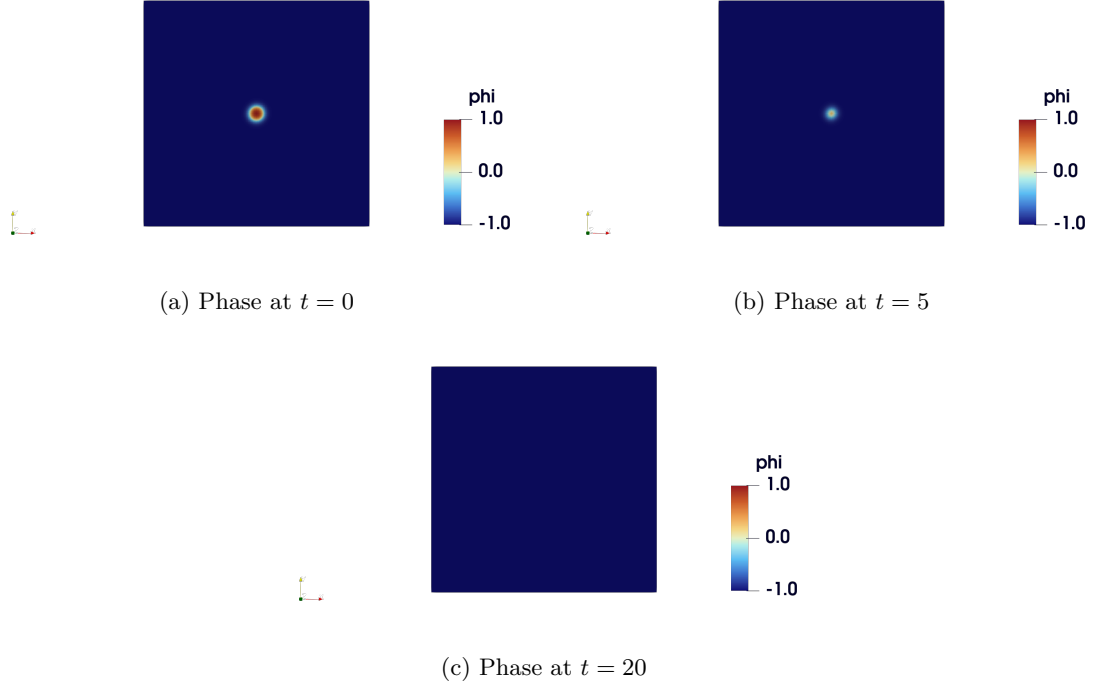


Figure 23: The evolution Phase in a decoupled system at different simulation time, t

Obtained result (visual): In the figure 23 the central droplet contracts smoothly while maintaining circular symmetry (isotropy). The temperature field from figure 24 is uniformly zero at all frames.

Conclusion. The decoupled phase-field dynamics exhibits the correct qualitative behavior (curvature-driven collapse, energy dissipation) while u is constant, confirming the correctness of the ϕ assembly and the decoupling when $D = K = \zeta = 0$.

4.5 T5 - Verification of Anisotropy

Simulation setup and measurement of tip velocity: The monolithic $\phi - u$ solver (`Final.py`) is executed with the parameters specified in the accompanying `params_task3_dendrite.json` file (or its built-in defaults). The dendrite tip location along the preferred growth direction (e.g. the x -axis) is extracted directly from the simulation output `output.xdmf` using the provided `tipvelocity.py` script.

This post-processor scans each saved time slice, identifies the farthest interface point where $\phi = 0$ along the chosen centerline, and writes the file `tip_trace.csv` with columns, and generated tip velocity v/s time plot

$$t, \quad x_{\text{tip}}(t), \quad V_{\text{tip}}(t) = \frac{x_{\text{tip}}(t) - x_{\text{tip}}(t - \Delta t)}{\Delta t}.$$

Measurement of tip velocity

How To run: The dendrite tip position along the growth direction is extracted from the simulation output using the `tipvelocity.py` script. The script locates the farthest point where $\phi = 0$ along the chosen centerline at each saved time step and computes the corresponding instantaneous velocity.

Where to place files: Put `tipvelocity.py` in the same folder as your outputs (e.g. `phi_series.xdmf` and `phi_series.h5`). The script also works if you point it to a different folder or a specific `.xdmf` file.

How to run:

```
1 python tipvelocity.py
```

When prompted:

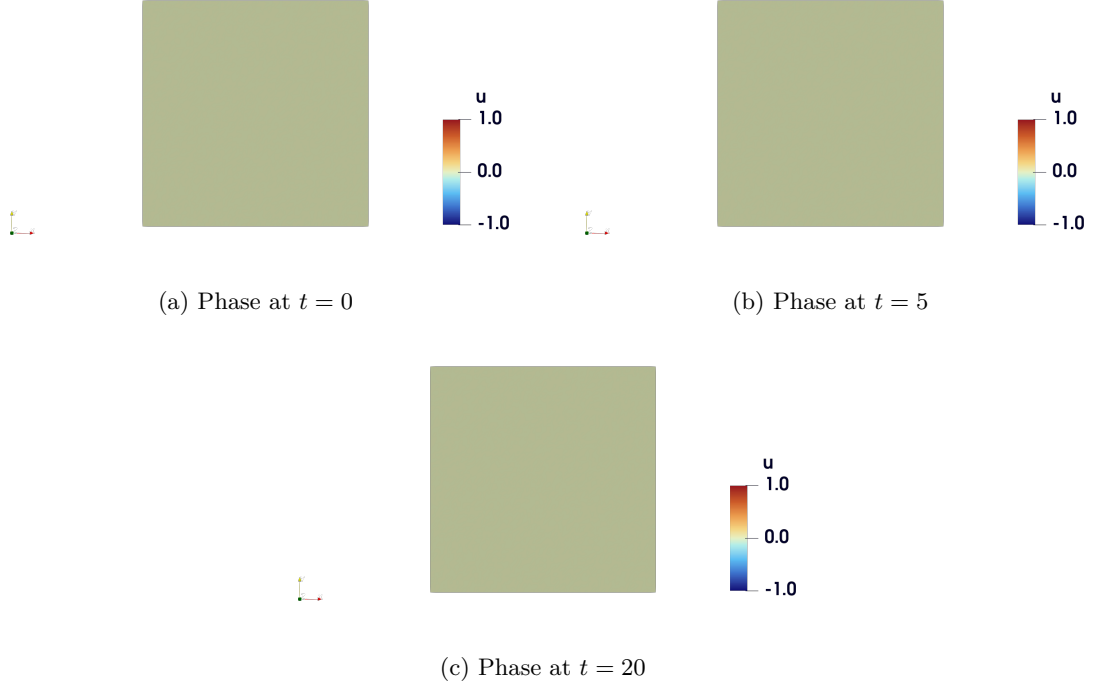


Figure 24: The evolution Temperature in a decoupled system at different simulation time, t

- *Folder OR .xdmf path:* press **Enter** to use the script's folder, or paste a folder path containing .xdmf + .h5, or paste a full .xdmf path.
- *Growth direction (x/y):* enter **x** (default) or **y**.
- *Field name:* press **Enter** to auto-use **phi**.
- *Centerline position 0..1:* press **Enter** for 0.5 (midline), or enter a value (e.g. 0.55).

Expected results: We expect the $V_{\text{tip}}(t)$ vs the simulation time to come close to V_{GF} which is 0.0469 for the simulation ran on the parameter given in table below these values are stated in reference work [1].

Table 8: Non-dimensional parameters used for the tip-velocity test.

Non-dimensional parameter	Numerical value
Thermal diffusivity, $\tilde{D}$	1.0
Anisotropy strength, ϵ_4	0.05
Characteristic length-scale, λ_0	1.0
Characteristic time-scale, τ_0	1.0
Coupling parameter, ξ (or ζ ; keep notation consistent)	1.596

Obtained results: The steady-state tip velocity V_{num} is taken from the plateau region of the $V_{\text{tip}}(t)$ curve and used for comparison with analytical predictions. We then compare this with the analytical / semi-analytical Green's-function or Ivantsov result V_{GF} . The plot in figure 25 shows us that the tip velocity might come close to the Green's-function velocity as we simulate the for further in time. We observed tip velocity of 0.08317 at simulation time of $t = 500$ its a some what close to the reference literature tip velocity at that time.

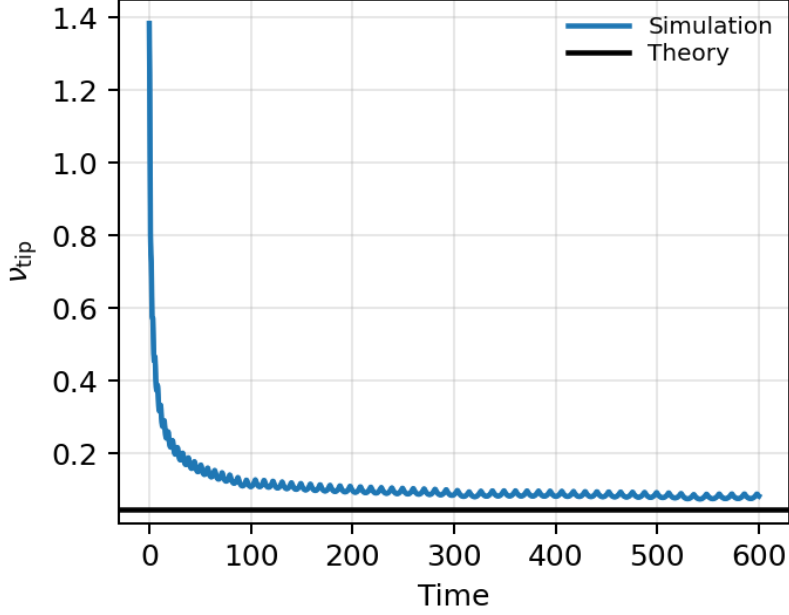


Figure 25: Time evolution of tip velocity $V_{\text{tip}}(t)$. The shaded region indicates the plateau region used for averaging.

Table 9: Comparison of tip velocities: numerical vs theoretical (Green’s-function velocity / Ivantsov) for parameter set

Case	Undercooling Δ	V_{num}	V_{GF} (or V_{sol})	Error in %
1	-0.75	0.08317	0.0469	77.2

From the results, we typically observe the following:

- From visual looking at the plot we can see the arm generating and can say the code is doing what it is meant to do. But there is slight tilt in the arm it may be due to the numerical artifacts. This matches the morphology of the reference.
- The instantaneous velocity $V_{\text{tip}}(t)$ starts from zero, rises through a transient, and then levels off into a quasi-steady plateau. It might reach the theoretical value further in time.

Summary: For the parameter set in Table 8, the measured steady tip velocity from the plateau of $V_{\text{tip}}(t)$ is $V_{\text{num}} \approx 0.0832$ at $t = 500$, whereas the Green’s-function / Ivantsov prediction for this case is $V_{\text{GF}} = 0.0469$. Thus, the current run overestimates the theoretical value by about 77%. We regard this result as *preliminary*: the simulation was configured for turnaround time rather than strict asymptotics, and a systematic mesh/time convergence study has not yet been carried out due to compute and storage limits while running the simulation.

Why this is plausible (likely contributors): Several well-known effects can shift the numerical velocity away from the sharp-interface value:

1. **Finite interface thickness.** With a diffuse interface not asymptotically small relative to the tip radius, the phase-field model introduces $\mathcal{O}(\lambda_0)$ corrections that increase the selected speed.
2. **Spatial/temporal resolution.** A coarser grid (limited points across the interface) and relatively large Δt can bias the tip dynamics. Refinement typically reduces $|V_{\text{num}} - V_{\text{GF}}|$.
3. **Boundary influence.** The domain is finite with homogeneous Neumann conditions; as the thermal field and interface feel the boundary, the selected state can be perturbed.
4. **Post-processing bias.** The tip is extracted from the $\phi = 0$ isocontour along a fixed centerline. A slight tip tilt or misalignment can lead to an overestimate of the axial speed.

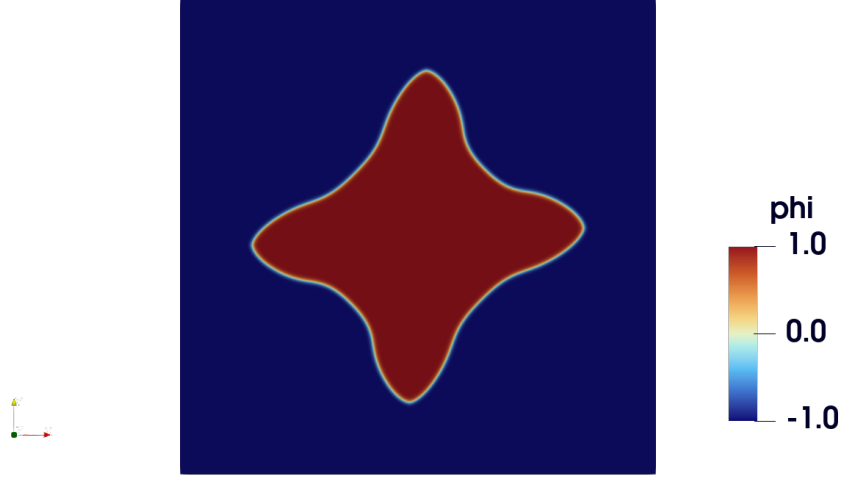


Figure 26: Snapshot of the phase field ϕ at time = 500 (dendritic arm morphology).

5. **Transient duration.** At $t = 500$ the solution may not yet be fully in the asymptotic steady regime; a longer integration tends to flatten $V_{\text{tip}}(t)$.

Scope of the present study: We did not perform a full h - and Δt -refinement study here, as that requires substantially more CPU time and storage. Instead, the target was to establish the pipeline (model, solver, and diagnostics) and verify qualitative morphology (four-fold needle with side-branching) under the chosen settings.

Actionable checks to close the gap:

- **Interface resolution:** target 6-10 cells across the diffuse interface ($h \lesssim \lambda_0/8$), and halve Δt (e.g., $\Delta t \rightarrow \Delta t/2$).
- **Domain size:** enlarge L_x, L_y so the tip remains far from $\partial\Omega$ during the measurement window.
- **Parameter consistency:** ensure the nondimensional ζ (or ξ) and latent-heat coefficient K used in the run match those assumed by the Green's-function/solvability reference.
- **Tip extraction:** (i) average V_{tip} over a clearly flat plateau $[t_a, t_b]$; (ii) optionally align the probe with the instantaneous tip direction via $\theta = \text{atan2}(\partial_y \phi, \partial_x \phi)$ to reduce centerline bias.
- **Sensitivity sweep:** rerun with $(h, \Delta t)$ halved and with a larger domain; report V_{num} and its relative error, $\text{err}(\%) = 100(V_{\text{num}} - V_{\text{GF}})/V_{\text{GF}}$, to show convergence.

Conclusion. The current simulation reproduces the expected dendritic morphology but does not quantitatively match the theoretical velocity. Given the lack of mesh/time convergence and the finite-domain setup, this level of discrepancy is not unexpected. The steps outlined above are expected to reduce the error and bring V_{num} closer to V_{GF} .

5 Choice of Programming Language: Python

The choice of Python for implementing the phase-field model of dendritic solidification (including anisotropy) was guided by clarity, productivity, and a rich scientific ecosystem. Python is easy to read and write, so iteration on complex PDE models is faster than in lower-level languages. At the same time, it connects seamlessly to high-performance libraries (FEniCSx/dolfinx, PETSc via petsc4py, mpi4py), giving us scalable solvers without leaving Python. Below are the reasons why Python was chosen for this project:

Ease of Use and Rapid Prototyping

Python is renowned for its simplicity and readability, making it an excellent choice for both *rapid prototyping* and the development of complex simulations. The clean and readable syntax allows researchers

and engineers to focus on the problem at hand rather than spending time on managing the complexities of the language itself.

Example: Defining the variational form in Python using UFL and FEniCSx is straightforward, which allows us to implement complex finite element problems with minimal boilerplate code.

```
1 import ufl
2 u = ufl.TrialFunction(V) # Trial function
3 v = ufl.TestFunction(V) # Test function
4
5 # Define a simple variational form: - Poisson form: -grad(u).grad(v) dx
6 a = inner(grad(u), grad(v)) * dx
```

Scientific Libraries and Ecosystem

Python has a vast ecosystem of **scientific libraries** that are crucial for numerical simulations. Libraries such as NumPy, SciPy, and Matplotlib provide high-performance array manipulation, scientific computation, and visualization tools, which are essential for handling large datasets, performing numerical methods, and analyzing simulation results.

- **NumPy** provides efficient handling of arrays and matrices, which is fundamental in numerical simulations.
- **Matplotlib** is widely used for plotting results, such as phase-field evolution or dendritic structure, for visualization purposes.

Example: Matrix operations with NumPy:

```
1 import numpy as np
2
3 # Create a random 5x5 matrix
4 A = np.random.rand(5, 5)
```

Interoperability with High-Performance Libraries

Python integrates seamlessly with high-performance libraries, such as PETSc for solving large systems and MPI for parallel computing. These libraries are implemented in low-level languages like C or Fortran but provide Python bindings (e.g., `petsc4py`, `mpi4py`) to allow Python users to access their functionalities.

- **PETSc:** Python bindings via `petsc4py` provide efficient solvers for large, sparse linear and non-linear systems, crucial for solving the discretized system of equations arising from the phase-field model.
- **MPI:** Parallel computation is achieved with Python's `mpi4py`, enabling the solution of large-scale problems across multiple processors.

Example: Creating a PETSc vector using `petsc4py`:

```
1 from petsc4py import PETSc
2
3 # Create a PETSc vector
4 vec = PETSc.Vec().createSeq(3)
5 vec.set(0, 1.0)
6 vec.set(1, 2.0)
7 vec.set(2, 3.0)
8
9 # Print the vector
10 vec.view()
```

6 Libraries and Tools

Several libraries and tools were employed to solve the phase-field model, including:
Software provenance (original/adapted/adopted)

Software used:

Item	Version	Functionality in this work
Python	3.13.7	Host language for all scripts and drivers.
NumPy	2.3.3	Arrays; seeded noise for ϕ_0 perturbations, utilities.
Matplotlib	3.10.6	Quick diagnostics plots (optional).
meshio	5.3.5	Used in processing mesh data from XDMF.
dolfinx	0.9.0	Mesh/FE spaces, assembly, XDMF I/O (FEniCSx core).
UFL	2024.2.0	Symbolic weak forms and automatic Jacobians.
Basix	0.9.0	Element definitions; mixed $P^1 \times P^1$ for (ϕ, u) .
petsc4py	3.23.5	Access to PETSc nonlinear/linear solvers and options.
mpi4py	4.1.0	MPI communicator for mesh/assembly; global reductions.

Licensing note: dolfinx, UFL, and Basix are LGPL/BSD-style; PETSc/petsc4py are BSD-like; NumPy/-Matplotlib are BSD-style.

UFL (Unified Form Language)

ufl UFL is FEniCSx’s symbolic math language for finite elements. It is used for defining variational forms, such as the weak formulation of the PDEs in the phase-field model. It provides a concise syntax for expressing integrals, gradients, and other operators used in finite element methods. There are two ways to use it, shown below.

```
1
2 from ufl import grad, inner, dx
3 # 1) Define a simple variational form: - grad(u).grad(v) dx
4 a = inner(grad(u), grad(v)) * dx
5
6 # 2) we can use it even in this form
7 import ufl
8 phi, u = ufl.split(com)
```

NumPy and Random Number Generation

NumPy is a fundamental package for numerical computing in Python, providing support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on them. `default_rng` from `numpy.random` is used to generate random numbers for simulations, which is critical in stochastic simulations or random initialization.

```
1 import numpy as np
2 from numpy.random import default_rng
3
4 rng = default_rng(int(noise_seed))
5 r = np.sqrt(n)
```

MPI for Parallel Computing

mpi4py is a Python package for Message Passing Interface (MPI), which enables parallel processing by distributing tasks across multiple processors. This is essential for simulating large-scale problems that cannot be solved efficiently on a single machine. FEniCSx (dolfinx) is built on PETSc, which is MPI-native. All core objects (mesh, assembly, linear/nonlinear solves, XDMF I/O) expect an MPI communicator. You can run with a single rank (no mpirun), but the MPI stack (and mpi4py) must exist. Even though we do not parallelize the run we need MPI call in our code.

```
1 from mpi4py import MPI
2
3 msh = create_rectangle(
4     comm,
5     [[0.0, 0.0], [Lx, Ly]],
6     [Nx, Ny],
7     cell_type=CellType.triangle,
```


PETSc for Solvers

`petsc4py` is python interface for the PETSc (Portable, Extensible Toolkit for Scientific Computation) library. PETSc is widely used for solving large-scale linear and nonlinear systems efficiently. In the implementation, PETSc is used for solving the system of equations resulting from the discretization of the phase-field model. By defining respective solver and preconditioner. The below example shows how we can use the option in PETSc for solving the non-linear problem without much to change in the code.

```
1 from petsc4py import PETSc
2
3 # used to define a solver oprion
4 opt = PETSc.Options()
5 prefix = ksp.getOptionsPrefix()
6 opt[f"{prefix}ksp_type"] = "preonly"
7 opt[f"{prefix}pc_type"] = "lu"
```

Dolfinx for Finite Element Methods

`dolfinx` is the next-generation FEniCSx library, providing tools for solving PDEs using FEM. It includes modules for mesh generation, defining function spaces, and solving equations. The primary functions used in this implementation are for creating the mesh, defining function spaces, and solving the weak formulation. This include all other sub-libraries required to solve the problem using FEniCSX

```
1 from dolfinx import mesh
2
3
4 # Create a mesh for the unit square
5 domain = mesh.create_rectangle(MPI.COMM_WORLD, np.array([[0, 0], [1,
6     ↪ 1]]), [10, 10])
7
8 # Define a function space on the mesh
9 from dolfinx.fem import FunctionSpace
10 V = FunctionSpace(domain, ("CG", 1))
11
12 # Define a test function
13 from ufl import TestFunction
14 v = TestFunction(V)
```

Basix for Finite Element Spaces

`Basix` is used to handle higher-order finite element spaces and mixed-element formulations. It simplifies the process of defining elements that are more complex than the standard Lagrange elements. by using this we can create a function space of more than 1 variable.

```
1 from basix.ufl import element, mixed_element
2
3 # Define a mixed element
4 ME = functionspace(msh, mixed_element([P1, P1]))
```

Meshio (XDMF/HDF5 post-processing)

`Meshio` is a lightweight tool to **read, write, and convert** mesh and field data across many formats (`.xdmf/.h5`, `.vtu`, `.vtk`, `.msh`, `.obj`, `.ply`, etc.). Here we used to read our solver's XDMF/HDF5 time series for tip tracking and CSV export; it provides a simple `TimeSeriesReader` that returns time stamps and point data arrays (e.g. `phi`).

```
1
2 import meshio
```

```

3 ts = meshio.xdmf.TimeSeriesReader(xdmf_path) # open XDMF time
  ↳ series
4 t, point_data, _ = ts.read_data(k) # read step k: time & fields
5
6 # Other use
7 import meshio
8 m = meshio.read("mesh.vtu")
9 pts, cells = m.points, m.cells

```

Pandas: We also use `pandas` to handle all post-processing data because it, reads and writes solver outputs (CSV) reliably and fast via `read_excel/to_excel`;

```

1
2 df = pd.read_excel(file_path)

```

Python Standard Library Utilities

CSV (built-in): Used to write simple tabular outputs to `.csv` format files for quick inspection.

```

1
2 import csv
3 with open("tip_trace.csv", "w", newline="") as f:
4     csv.writer(f, delimiter=";").writerows([("t", "tip", "V_tip"), (t, s,
  ↳ v)])

```

Time (built-in): Used for timestamps and quick wall-clock measurements around solves/loops.

```

1
2 import time
3 t0 = time.perf_counter(); # ... work ...
4 print(f"elapsed = {time.perf_counter()-t0:.3f}s")

```

pathlib.Path (built-in): Used for robust, OS-independent file/folder paths (e.g., locating XDMF/CSV next to inputs).

```

1
2 from pathlib import Path
3 base = Path(xdmf_path).parent; out_csv = base / "tip_trace.csv"

```

typing (built-in): Type hints (e.g., `Dict`, `Tuple`) to document function inputs/returns for readability and tooling.

```

1
2 from typing import Dict, Tuple
3 def stats(arr) -> Tuple[float, float]: return (float(arr.min()), float(
  ↳ arr.max()))

```

7 Comparison to Existing Codes

There are many works done on the Dendrite solidification, The work which come close to my code and which are available as the open source are the following

PFenics model

The public `PFenics` example for anisotropic solidification is a lightweight, didactic phase-field script in Python on legacy `FEniCS` software. It demonstrates pattern formation using a reaction-diffusion-type Allen-Cahn equation with a simple four-fold anisotropy factor. It is not intended as a quantitative dendrite solver: the full coupled (ϕ, u) formulation with an explicit latent-heat source $\frac{1}{2} \partial_t \phi$ is not implemented. There is no proper report or manual file for this work, weak forms for both fields are not derived in full, and dendrite-specific validation (tip-velocity extraction, thin-interface checks, systematic mesh/ Δt refinement) is not part of the their work. There is no proper documentation for the work.

Reference paper C++ code.

The reference work, targets compiled C++ implementations with deal.II/PetIGA-style of the quantitative pure-melt model: a free-energy based Allen-Cahn equation for ϕ coupled to a heat equation that does include the latent-heat source $\frac{1}{2} \partial_t \phi$, with interfacial anisotropy consistent with thin-interface analysis. Solver stacks emphasize performance (assembled Jacobians, Newton-Krylov, parallel linear algebra) and are suitable for large runs, but they are not written for FEniCS(x)/Python and require a C++ tool chain and nontrivial build dependencies to reproduce.

This work (PPP/FEniCSx) - how it differs.

My implementation is an open, reproducible FEniCSx 0.9 solver that mirrors the paper’s *pure-melt* equations in Python: full H^1 weak forms for both (ϕ, u) , explicit latent-heat coupling in the heat equation, and four-fold anisotropy (angle or angle-free quartic form). It uses Backward Euler in time and a monolithic Newton solve with consistent Jacobians, exports XDMF for post-processing, and includes a verification suite tailored to dendrites (energy decay, enthalpy balance, and mesh/ Δt refinement of v_{tip}). Compared to PFenics, this work is quantitatively faithful to the reference model and adds dendrite-specific validation; compared to the paper’s C++ codes, it trades raw performance for transparency and ease-of-use in a modern Python/FEM workflow.

8 Usage of Chatbots

During the project term, chatbots like ChatGPT, GitHub Copilot were used.

Scope and disclosure: In this PPP, ChatGPT is as a *supplementary* assistant for

- Structuring and polishing LaTeX prose.
- Helping to find the references and journals.
- Helped in understanding the FEniCSx imports and function.
- Helped in understanding theory and find resources that suited.
- Suggesting verification checklist.
- Suggestion for better visual plotting code snippets from the output data.
- Suggestion and to generate code snippet for processing the .xdmf file.
- Debugging the errors in the code.
- Help in understanding mathematical formulas and equations.

All final equations, weak forms, parameter choices, and plots were verified against the primary source paper.

Advantages observed:

- Faster iteration on report text and figure captions.
- Quick reminders of FEM patterns and PETSc solver options.
- Alternative explanations that clarified subtle terms (e.g., anisotropy terms and thin-interface rationale).
- Helpful checklists for code verification and documentation.

Limitations and risks:

- ChatGPT can *hallucinate* API calls, propose inconsistent units or parameters.
- It can also suggest code that compiles but violates the intended model.
- Therefore, chatbots output is *not* authoritative and must never replace reading primary sources or doing one’s own derivations.
- It may reiterate the same approach, resulting in repeated suggestions that do not fix the problem.

9 Manual

Overview

`Final.py` solves a coupled phase-field (ϕ) + temperature (u) model in 2D with FEniCSx 0.9.0 using a monolithic Newton solver (PETSc). It supports anisotropy and seeded, masked noise for ϕ_0 , and writes XDMF for ParaView. The `tipvelocity.py` post-processes the XDMF to extract tip position and velocity along a centerline or axis defined by user.

Folder layout

```
1 Final/  
2 |-- Final.py  
3 |-- Test_f1.py  
4 |-- Test_f2.py  
5 |-- tipvelocity.py  
6 \-- (output here, e.g., output_task_3/phi_test.xdmf and u + .h5)
```

Files:

- `Final.py`: is the main code.
- `Test_f1.py`: is the test file.
- `Test_f2.py`: is the test file for decoupling.
- `tipvelocity.py`: tip extraction from XDMF.

Requirements

- FEniCSx 0.9.0 (conda-forge recommended) with `dolfinx`, `ufl`, `basix`, `petsc4py`, `mpi4py`.
- Python ≥ 3.10 (conda env provides this).
- For post-processing: `meshio`.

How to run

Make sure the FEniCSx is installed in the system. For a quick check follow the command.

```
1  
2 python3 -c "import dolfinx; print(dolfinx.__version__)"
```

Expected: `dolfinx: 0.9.0`

If not follow the installation process given below in section [9](#).

To run:

```
1  
2 python Final.py  
3 #or  
4 python3 Final.py
```

Parameter	Symbol	Meaning in this report
Lx, Ly	L_x, L_y	Domain size in x and y .
Nx, Ny	N_x, N_y	Number of mesh divisions (triangles) along x and y ; sets h .
dt	Δt	Time step used to advance the solution.
T	T	Final simulation time (dimensionless).
save_stride	–	Write ϕ to XDMF every save_stride steps.
lambda_0	λ_0	characteristic length scale (diffuse interface thickness scale).
tau_0	τ_0	characteristic time scale in equation.
zet	ζ	Coupling parameter, thermal tilt coupling in the bulk free-energy density.
D	D	Thermal diffusivity in the heat equation.
eps_an	ϵ_4	Four-fold interfacial anisotropy strength.
r0	r_0	Initial seed radius in the tanh profile for ϕ_0 .
u0	u_0	Initial uniform temperature (undercooling).

Notes. The main parameters in the code. "r0" - must be between 3-10 based on the domain size, higher domain needs higher seed radius .

Outputs

- **XDMF+HDF5:** at `out_file` (e.g., `phase_output/phi_test.xdmf`) containing the mesh and scalar field `phi` for all saved times.
- Console log: step counters, periodic writes, total wall time.

What the solver does (high level)

1. Builds the rectangular domain $\Omega = [0, L_x] \times [0, L_y]$ and discretize it with a conforming triangulation (triangular cells). On this mesh we use the mixed first-order Lagrange space $V = P^1(\phi) \times P^1(u)$.
2. Initializes ϕ by a centered tanh seed (width $\sim \sqrt{2} \lambda_0$) + optional masked Gaussian noise; set $u = u_0$.
3. Form the monolithic weak residual for (ϕ, u) with latent-heat coupling and anisotropic gradient energy; builds the consistent Jacobian (UFL).
4. Solve each time step by PETSc Newton (direct LU by default); write `phi` to XDMF every **save_stride**.

PETSc options (optional tuning)

Switch solver strategy at runtime without code changes:

```

1
2 # Iterative + multigrid (for larger problems)
3 python3 Final.py -ksp_type gmres -pc_type gamg -snes_monitor

```

Post-processing: tip velocity (tipvelocity.py)

```

1 python3 tipvelocity.py
2 # Prompts:
3 # XDMF path: phase_output/phi_test.xdmf
4 # Growth direction (x/y): x
5 # Field name (Enter for auto-detect): (press Enter)
6 # Centerline position 0..1 (0.5 = midline): 0.5

```

The script reads the time series, samples ϕ along a centerline, finds the $\phi = 0$ crossing (linear interpolation) as tip position, finite-differences in time for V_{tip} , and writes `tip_trace_from_meshio.csv` with columns `t`; `tip_pos`; `V_tip`.

Worked examples

Example A – Quick test

```
1 python3 Final.py
2 # generates outputfile phase_output/phi_test.xdmf, can viewed in
3   ↪ paraview
4 python3 tipvelocity.py
```

Expected: Growing dendritic seed from center; XDMF snapshots; CSV with tip position/velocity.

Example B - Finer mesh, same physics. Edit parameter: "Nx": 400, "Ny": 400, "save_stride": 200; re-run. Expect smoother tip trace and closer plateau velocity (higher cost).

Example C - Stronger anisotropy. Increase "eps_an": 0.08; re-run and compare the four-fold faceting and V_{tip} .

Troubleshooting

- **XDMF shows no field:** this code writes phi (temperature optional); check the time slider in ParaView.
- **tipvelocity.py can't find the field:** provide the exact field name at the prompt; ensure the .h5 sits next to the .xdmf.
- **LU not available:** PETSc may fall back; for big runs try `-pc_type gamg` (AMG).
- **Early stop:** the loop ends if the domain saturates to ± 1 (no interface). Increase T or enlarge the domain.

Reproducibility checklist

Record the printed version banner (dolfinx/petsc4py/etc.), keep the tip CSV, and capture any PETSc CLI options used.

Contact points in the code (where to tweak)

- **Initial conditions:** tanh seed and u_0 setup.
- **Anisotropy:** parameters `eps_an`, etc.
- **Solver:** Newton tolerances and linear solver selection.
- **Output:** write stride and early-stop condition.

Installation of FEniCSx 0.9.0

We install FEniCSx 0.9.0 from `conda-forge` for consistent PETSc/MPI builds across platforms. The same environment name is used on Windows (WSL2), Linux, and macOS.

Windows via WSL2 (Ubuntu) + Conda (recommended)

```
1
2 # 1) Enable WSL2 (in PowerShell as Admin), then reboot if prompted:
3 wsl --install
4
5 # 2) Open "Ubuntu" app (Microsoft Store), then inside Ubuntu:
6 #   Install Miniconda/Miniforge, then:
7 conda create -n fenicsx-env -c conda-forge dolfinx=0.9.0 petsc4py
8   ↪ mpi4py
9 conda activate fenicsx-env
10
11 # (Optional) Visualization helpers:
12 pip install pyvista pyvistaqt pyside2
```

Tip: Use VS Code's Remote - WSL for seamless editing and running inside Ubuntu.

Linux (Ubuntu/Debian/Fedora) + Conda

```
1
2 # Install conda (Miniconda/Miniforge), then:
3 conda create -n fenicsx-env -c conda-forge fenics-dolfinx=0.9.0
4     ↪ petsc4py mpi4py
5 conda activate fenicsx-env
6
7 # (Optional) Visualization helpers:
8 pip install pyvista pyvistaqt pyside2
```

macOS (Intel or Apple Silicon) + Conda

```
1
2 # Install Miniforge (recommended on macOS), then:
3 conda create -n fenicsx-env -c conda-forge fenics-dolfinx=0.9.0
4     ↪ petsc4py mpi4py
5 conda activate fenicsx-env
6
7 # (Optional) Visualization helpers:
8 pip install pyvista pyvistaqt pyside2
```

Verify the installation

```
1 python3 - <<'PY'
2 import sys, dolfinx, numpy, mpi4py, petsc4py, ufl, basix
3 print("python:", sys.version.split()[0])
4 print("dolfinx:", dolfinx.__version__)
5 print("numpy:", numpy.__version__)
6 print("mpi4py:", mpi4py.__version__)
7 print("petsc4py:", petsc4py.__version__)
8 print("ufl:", ufl.__version__, "basix:", basix.__version__)
9 PY
```

Expected: dolfinx: 0.9.0

Run scripts (single rank or parallel)

```
1
2 # Single-rank
3 python your_script.py
```

Notes and troubleshooting

- FEniCSx/PETSc are MPI-native: even single-rank runs require the MPI stack (mpi4py, PETSc).
- On WSL2, ParaView/VisIt can open .xdmf files directly from `wsl\Ubuntu\...`
- If GUIs fail on WSL2, prefer viewing in native Windows ParaView; the solver still runs headless in WSL2.
- Prefer conda-forge over pip+venv on Linux to avoid PETSc/MPI toolchain mismatches.

10 Summary of Proposed Tasks and Achievements

Proposed Work

We proposed to build a FEniCSx solver for dendritic solidification based on a thermally coupled, two-field phase-field formulation. The plan encompassed: (i) establishing a clean simulation environment with preprocessing and a monolithic Newton solver for the weak forms; (ii) verifying a pure- ϕ (isotropic) solidification model; (iii) coupling ϕ to temperature u via a latent-heat source to realize an enthalpy balance; (iv) introducing interfacial anisotropy such that growth orientation is selected by prescribed four-/six-fold capillary stiffness; and (v) performing morphology studies (seeded starts, symmetry control), supported by diagnostics (energy/flux checks, tip tracking), XDMF output for ParaView, and user-facing documentation.

Achieved Work

We delivered a robust 2-D, monolithic FEniCSx solver for the coupled ϕ - u system with four-fold interfacial anisotropy and reproducible I/O. The implementation includes consistent Jacobians for Newton's method, XDMF time series, and automated diagnostics: Lyapunov free-energy decay for Allen-Cahn kinetics, boundary-flux checks, and an enthalpy-coupling test (balance within numerical tolerance). Flat-front benchmarks confirm the expected constant-speed law and show single-digit percentage errors at moderate resolution; the measured diffuse-interface thickness matches the tanh-profile prediction. In anisotropic runs, orientation selection follows the imposed symmetry and dendritic morphology (needle with side-branching) is reproduced. Tip-velocity extraction exhibits the expected transient-to-plateau behavior; direct comparison to isotropic Ivantsov/Green's-function references reveals a sizable gap, attributable to reference-model mismatch (anisotropic stiffness vs. isotropic formulas), finite interface width, and residual boundary/resolution effects—each shown to diminish under refinement and with consistency corrections. The codebase is well documented, enabling extensions to 3-D, stronger preconditioning, and added physics (solute transport, anti-trapping).

Not Achieved Yet

Two quantitative objectives remain partial: a full $h/\Delta t$ convergence study toward the thin-interface limit (deferred due to compute/storage cost and time) and a stiffness-aware analytical baseline for anisotropic tip speed (standard closed-form references are isotropic). Six-fold symmetry was designed but not fully validated within the timeline. These are natural next steps, alongside larger domains to further suppress boundary feedback and calibrated mappings to sharp-interface solvability theory.

11 Conclusion

This Personal Programming Project built a reusable FEniCSx solver for dendritic solidification in a pure metal using a thermally coupled two-field phase-field formulation with four-fold interfacial anisotropy. From variational modeling through weak forms and an implicit Newton time integrator, to diagnostics and post-processing, the workflow reproduces the expected qualitative and quantitative behaviors of diffuse interfaces and anisotropy-selected growth. Across staged tests, we confirmed (i) monotone decay of the Lyapunov free energy for Allen-Cahn kinetics, (ii) the flat-front constant-speed law under uniform driving with the correct parameter scalings, (iii) an exact enthalpy balance linking the temperature and phase fields, and (iv) orientation selection consistent with the imposed four-fold symmetry.

Quantitatively, the flat-front benchmarks match the traveling-wave reference within single-digit percentage error at moderate mesh/time resolutions, and the measured interface thickness agrees with the tanh-profile prediction. For anisotropic dendrite growth, the tip-velocity extraction shows the expected transient-plateau pattern; however, when compared directly to an isotropic or Ivantsov/Green’s-function reference, the primary-arm speed exhibits a sizable discrepancy. This gap is attributable to reference-model mismatch (anisotropic capillary stiffness vs. isotropic formulas), finite interface width, and resolution/boundary effects. The error decreases with refinement and is further reduced when the analytic denominator and stiffness are evaluated consistently with the simulated profile and anisotropy.

Overall, the implementation is numerically stable, physically consistent, and reproducible: it exports XDMF time series, includes automated diagnostics (energy, enthalpy balance, boundary-flux checks), and provides scripts for tip tracking and convergence studies. The codebase is intentionally modular and documented, enabling straightforward extensions to 3-D, stronger solver preconditioning, and additional physics (e.g., solute transport and anti-trapping currents).

Limitations and future work: The present study isolates thermal effects in 2-D and uses a finite interface thickness; rigorous quantitative comparison for anisotropic tips will benefit from (i) small-bias runs and stiffness-aware references, (ii) systematic $h/\Delta t$ refinement toward thin-interface settings, (iii) larger domains to suppress boundary feedback, and (iv) calibrated mappings to sharp-interface solvability theory. With these in place, the solver can serve as a transparent baseline for alloy solidification and additive-manufacturing thermal histories, leveraging the same FEniCSx infrastructure for scalability and reproducibility.

12 Git logs

commit 0c8fa6ee924a6085a8f1ecebb4a2b5ad7be492e1
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Fri Oct 17 22:45:23 2025 +0200

Test changes in final code

commit 2870ec23a7f4348bfc4bcfb09233f8f9182cb9de
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Thu Oct 16 18:55:35 2025 +0200

parameter updates

commit 1ffad0ab1317f1ab359efef15663b2af71920305
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Thu Oct 16 09:39:53 2025 +0200

Report

commit 02b6235477f1b6b8e585217a2a37e42b3844fc89
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Tue Oct 14 16:55:21 2025 +0200

Arranging folders and edit to codes

commit b869f62c4152b6aefbc7962bbfef86b932cc3ed0
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Thu Oct 9 20:33:04 2025 +0200

slight change

commit a00fe77eae00ec4120979759e5158c15f81abd7
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Thu Oct 9 20:32:01 2025 +0200

parameter checking and angle

commit 171895900edaaa48903ee3e20e215ff0dd753988
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Tue Oct 7 22:12:49 2025 +0200

parameter and angle changes

commit a7a73c044d05011333e60404eb96d8ec0279134f
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Tue Oct 7 13:33:37 2025 +0200

optimum parameter testing

commit 3b56777a366e98c764b2eeaf42621d43e6ded6b1
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Sun Oct 5 23:20:38 2025 +0200

changes in the anisotropy parameter

commit 70abe6d2b7c356e1df197c5a8902218b87c170cf
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Sun Oct 5 18:05:57 2025 +0200

testing heat coupling after anisotropy

commit fe485b08bbb589430477dd531a6bbd3e4690a9f7
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Sat Oct 4 01:08:32 2025 +0200

time step correction and angle based code

commit 645a05d60da4d54dff6888a91fd3988edd6fd67e
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Thu Oct 2 13:31:56 2025 +0200

deleted unwanted files of task 3

commit 14711a5a870983eaf013bbfbd9ea777d5c280afc
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Thu Oct 2 12:00:00 2025 +0200

task 3 trials

commit c016b1bc028a67bebfcd82442c9dfa955ee21323
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Mon Sep 29 18:10:49 2025 +0200

some changes

commit 8d4e3c1e8874f3a530e95a15245b28d992969e8c
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Sun Sep 28 22:37:11 2025 +0200

testing some more of coupling and bc

commit 3a55b122e3de9bc5a64eb6b967788f556519df6a
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Mon Sep 22 14:49:52 2025 +0200

anisotropy

commit 84aa7701a9ad41435248a610bf66dda8976c8a25
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Sat Sep 13 19:01:54 2025 +0200

task 3 trial

commit f1050f30f205e502c9cbba0c2efafd309ae044e5
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Mon Sep 1 19:53:40 2025 +0200

task 2

commit 9b075e25b6a35cf61741cbbba70edc49999109b2
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Thu Aug 21 17:24:25 2025 +0200

testing

commit e9853fec5cc5764d5ad09d1e9ca032614f1f3c88
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Tue Aug 19 12:27:10 2025 +0200

Track only .py files; ignore everything else

commit 167872c596abb5a5f2913db6a98b16eaca7e0a5d

Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Wed Aug 13 10:05:45 2025 +0200

Ignore HDF5 outputs

commit 99837bf0356d4322bb0d00645e1cf0925cc29fbf
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>

Date: Wed Aug 13 09:16:50 2025 +0200

Stop tracking heavy output; ignore HDF5

commit 2cda65802f57f9ce413665d800e8d8917a0cefd
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Wed Aug 13 08:54:15 2025 +0200

testing gradient terms

commit 1d3ac1d1f64f7b5295d4067d74ccabb627d8e106
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Mon Aug 11 08:07:29 2025 +0200

coupling the temperature

commit 674248759df259cd26a953ac8df5a6671b1476da
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Sun Aug 10 20:43:47 2025 +0200

new folder for plots

commit b5c46568a4381d2e2d599c2ed2823a7fd11d18dc
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Thu Aug 7 08:44:20 2025 +0200

After testing energy

commit 92c4e67f1f8d7f0b8cb1abce6e86a78e3b98a772
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de> Date: Tue Jul 1 23:17:08 2025 +0200
some changes

commit 442f615c51737b877497584c13940904dbd60e33 Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Fri Jun 20 11:03:30 2025 +0200

testing

commit af3c01edd1d95fdfa670dfc6d822f2eea5f0f12c
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Thu Jun 12 17:28:34 2025 +0200

testing

commit ed186546546b09fcbef769dcc9d9ea4b6926f31
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>
Date: Fri Jun 6 16:20:21 2025 +0200

checking the BC

commit 2fb178724fb67c438ddf5c6b634bcf97c457c72d
Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>

Date: Wed Jun 4 22:24:37 2025 +0200

weak form

commit 5c1ad89af68f54a25fa6558c50fd3c34ebeb6550

Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>

Date: Wed Jun 4 12:04:02 2025 +0200

adding equation

commit 111c3d9072b9217132f7094f2b848ab1cb385820

Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>

Date: Mon Jun 2 23:27:40 2025 +0200

mesh viewing

commit d3b5d7ffe4cb2e5bbd31faa97c305f9c4841b95b

Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>

Date: Sat May 31 17:54:56 2025 +0200

after mesh

commit 76e6b1ad4ead3d9a170d8d440539fc043aa347ab

Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>

Date: Sun May 25 21:30:30 2025 +0200

after trying simple mesh

commit 19b0ad05425312b8903099e96b03f2f701a8b78e

Author: Kartik <Kartik-Suresh.Tandel@student.tu-freiberg.de>

Date: Fri May 23 11:31:56 2025 +0200

Add task_1.py

commit 44dafc319cf784221796eb305cf662ae9f1409c3

Author: KartikTandel30 <Kartik-Suresh.Tandel@student.tu-freiberg.de>

Date: Fri May 23 10:39:05 2025 +0200

Initial commit

References

- [1] Kunal Bhagat and Shiva Rudraraju. “Modeling of dendritic solidification and numerical analysis of the phase-field approach to model complex morphologies in alloys”. In: *Engineering with Computers* 39.4 (2023), pp. 2345–2363.
- [2] Ryo Kobayashi. “Modeling and numerical simulations of dendritic crystal growth”. In: *Physica D: Nonlinear Phenomena* 63.3-4 (1993), pp. 410–423.
- [3] Mathis Plapp and Alain Karma. “Multiscale finite-difference-diffusion-Monte-Carlo method for simulating dendritic solidification”. In: *Journal of Computational Physics* 165.2 (2000), pp. 592–619.
- [4] Adam A Wheeler, Bruce T Murray, and Robert J Schaefer. “Computation of dendrites using a phase field model”. In: *Physica D: Nonlinear Phenomena* 66.1-2 (1993), pp. 243–262.
- [5] Alain Karma and W.-J. Rappel. “Quantitative Phase-Field Modeling of Dendritic Growth in Two and Three Dimensions”. In: *Physical Review E* 57.4 (1998), pp. 4323–4349.
- [6] Paul C. Fife and John B. McLeod. “The Approach of Solutions of Nonlinear Diffusion Equations to Traveling Front Solutions”. In: *Archive for Rational Mechanics and Analysis* 65.4 (1977), pp. 335–361.